INSAT · RÉSEAUX & TÉLÉCOMMUNICATIONS

Architectures Réactives et Microservices

Résumé de révision pour le DS : l'essentiel du cours — définitions, propriétés, formules, méthodes, schémas, exemples et exercices corrigés

1 **Patrons de conception (Design Patterns)**
GoF et classification des 23 patrons · Fabrique · Singleton · Adaptateur · Composite · Commande · Observateur · Stratégie · Itérateur

2 **Principes de conception orientée objet**
7 risques d'une mauvaise conception · SOLID · cohésion REP/CCP/CRP · couplage ADP/SDP/SAP · métriques I, A, D

3 **Architecture logicielle**
Définition · design vs architecture · couches et niveaux · BF/BNF · MoSCoW · scénarios de qualité · sync/async · tendances 2026

\+ **Annexes**
Fiche express à savoir par cœur · liens entre chapitres · QCM corrigé · sujets type DS corrigés

D'après le cours de Dr. Asma Ben Hassouna

Sommaire

Avant de commencer · Comment utiliser ce résumé 3

Chapitre 1 · Patrons de conception (Design Patterns) 4

1.1 Définition, 4 éléments et catalogue GoF 4
1.2 Patrons de création · Singleton 5
1.3 Fabrique (Factory) 6
1.4 Patrons structurels · Adaptateur (Adapter) 8
1.5 Composite 9
1.6 Patrons comportementaux · Itérateur, Stratégie 10
1.7 Commande (Command) 11
1.8 Observateur (Observer) 12
1.9 Synthèse et à retenir pour le DS 13
1.10 Exercices corrigés 14

Chapitre 2 · Principes de conception orientée objet 16

Pourquoi des principes ? Les 7 risques de conception 16
SOLID : vue d'ensemble 17
S et O — Responsabilité unique (SRP), Ouvert/fermé (OCP) 17
L et I — Substitution de Liskov (LSP), Ségrégation des interfaces (ISP) 18
D — Inversion des dépendances (DIP) 19
Cohésion des packages : REP, CCP, CRP 20
Couplage des packages : ADP, SDP, SAP 21
Métriques de Martin : I, A, D et séquence principale 22
À retenir pour le DS 24
Exercices corrigés 24

Chapitre 3 · Architecture logicielle 27

ShopExpress et définition de l'architecture 27
Architecture vs design : les 4 critères de décision 28
Pourquoi une architecture ? Bénéfices, cycle de vie, architecte 28
Niveaux d'abstraction : 4 couches, 5 niveaux, familles de produits 29
Découplage : trois versions du même système 30
Exigences : BF / BNF, MoSCoW et attributs de qualité 31
Scénarios de qualité : rendre un BNF testable 32
Compromis : pas d'architecture parfaite, synchrone vs asynchrone 33
Tendances 2026 : monolithe modulaire, événementiel, AI-ready 34
À retenir pour le DS 35
Atelier télémédecine et exercices corrigés 36

Annexes · Fiches de révision et sujets type DS 38

Fiche express : à savoir par cœur 38
Correspondances entre les chapitres 40
QCM éclair (corrigé) 41
Sujets transversaux type DS (corrigés) 41

AVANT DE COMMENCER

Comment utiliser ce résumé

Ce résumé condense les **trois chapitres** du cours *Architectures Réactives et Microservices* en suivant l'ordre et le vocabulaire des slides. Les trois chapitres décrivent **le même logiciel à trois niveaux de zoom** : les patrons à l'échelle de quelques classes, les principes à l'échelle des classes et des packages, l'architecture à l'échelle du système.

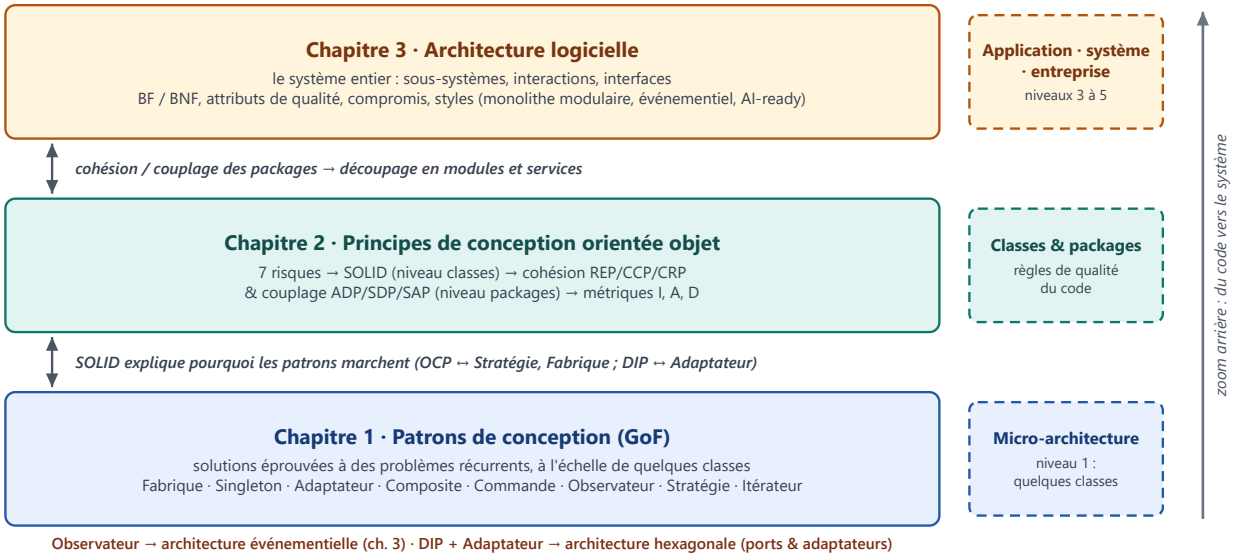


Figure 0.1 — Carte du cours : chaque chapitre éclaire un niveau de zoom différent du même logiciel.

Légende des encadrés

DÉFINITION
À connaître par cœur, souvent mot pour mot.

PROPRIÉTÉ / RÈGLE
Caractéristiques, conséquences, règles.

MÉTHODE
Démarche pas à pas pour une question de DS.

EXEMPLE CORRIGÉ
Cas concret du cours, expliqué.

POINT DS
Notion typique d'une question d'examen.

PIÈGE
Erreur fréquente ou **erreur d'une slide**, corrigée ici.

ASTUCE
Mnémotechnique, lien avec un autre chapitre.

Proposition de corrigé — non fournie dans le cours
: solution cohérente avec le cours, mais pas officielle.

CE QUI REVIENT LE PLUS SOUVENT AU DS

- **Ch. 1** : définition et 4 éléments d'un patron ; classer un patron (objectif × portée) ; reconnaître un patron dans un code ou un UML et nommer ses rôles (Target/Adaptee/Adapter, Subject/Observer, Invoker/Command/Receiver…) ; écrire un Singleton thread-safe et expliquer `static` et `synchronized`.
- **Ch. 2** : nommer le principe SOLID violé par un code et le corriger ; les 7 risques ; les correspondances CCP ↔ SRP et CRP ↔ ISP ; casser un cycle (ADP) ; calculer `I = Ce/(Ca+Ce)`, `A = Na/Nc`, `D = |A+I−1|` et interpréter (zone de douleur, zone d'inutilité).
- **Ch. 3** : définition de l'architecture (3 éléments) ; design ou architecture (4 critères, « un seul oui suffit ») ; BF vs BNF, Blanche-Neige ; MoSCoW ; scénario de qualité en 6 éléments ; synchrone / asynchrone ; les 3 modèles de découplage de BDD ; « il n'y a pas d'architecture optimale universelle ».

Méthode : ① comprendre quel problème résout chaque patron ou principe ; ② apprendre les encadrés *Définition* et la fiche express (annexes) ; ③ refaire les exercices corrigés sans regarder la correction.

CHAPITRE 1

Patrons de conception (Design Patterns)

Objectifs

- Définir un patron et ses **4 éléments** ; connaître les **23 patrons GoF** et les classer (objectif × portée).
- Maîtriser **Singleton, Fabrique, Adaptateur, Composite, Commande, Observateur** (+ Itérateur, Stratégie) : intention, UML, code Java, déroulement.

Ce qui tombe au DS

- Définitions mot pour mot ; classer un patron ; rôle de `static` / `synchronized`.
- Nommer les rôles d'un patron dans un UML ou un code ; prédire une **sortie console** ; choisir le patron d'un scénario.

Complément = hors diapositives. **Proposition de corrigé — non fournie dans le cours** = corrigé rédigé pour ce résumé. Les erreurs des diapositives sont corrigées dans les encadrés « Piège ».

1.1 Définition, 4 éléments et catalogue GoF

DÉFINITION (COURS)

Un **patron de conception** (*Design Pattern*) est une **solution générale et réutilisable** à un problème **qui revient souvent lors de la conception d'un logiciel**. Ce n'est **pas** une conception finie que l'on transforme directement en code : c'est une **description**, un **modèle** de résolution utilisable dans de nombreuses situations, une **recette** que l'on réutilise pour résoudre un problème connu **sans repartir de zéro**.

Les 5 apports (cours) : ① solutions **générales**, documentées indépendamment d'un problème particulier ; ② développement **accéléré** (paradigmes testés et éprouvés) ; ③ **expérience** de développeurs chevronnés ; ④ **problèmes subtils** anticipés ; ⑤ **lisibilité** : code clair et maintenable pour les codeurs et les architectes (le cours oppose `if (a && b && c)` à `if (estUtilisateurValide())`). Un patron sert aussi de **vocabulaire commun** (« on met un *Observer* ici »).

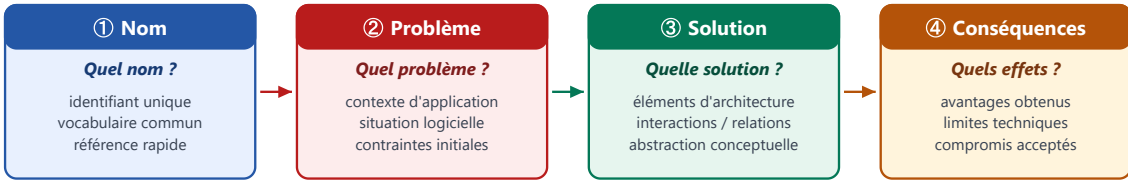


Figure 1.1 — Les 4 éléments d'un patron. En résumé (cours) : Nom → Quel problème ? → Quelle solution ? → Quels effets ?

EXEMPLE CORRIGÉ — DÉCRIRE LE SINGLETON AVEC LES 4 ÉLÉMENTS

Nom : Singleton (création, portée objet). **Problème** : une classe (journal, configuration) doit avoir *une seule* instance, accessible depuis tout le code, même avec plusieurs threads. **Solution** : constructeur `private`, attribut `private static` qui garde l'instance, méthode `public static synchronized getInstance()`. **Conséquences** : **+** instance unique, accès contrôlé ; **–** état global, tests plus difficiles, coût de la synchronisation.

Le catalogue GoF et ses deux critères de classement

La **Bande des quatre** (*Gang of Four*) : **Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides**, auteurs de *Design Patterns : Elements of Reusable Object-Oriented Software* (1994). Ils y définissent **23 patrons**, des **solutions éprouvées** à des problèmes récurrents. **Piège** : la diapositive légende la photo de Gamma « (Booch) » ; **Grady Booch** n'a écrit que la *préface*.

① Par OBJECTIF (ce que fait le patron)	
Création	Concerne le **processus de création** d'objets et de classes.
Structure	Concerne la **composition** des classes et des objets.
Comportement	Caractérise l'**interaction** et la **responsabilité** des objets et des classes.

② Par PORTÉE (sur quoi il agit)	
Classe	Relations classes / sous-classes (**héritage**), définies **statiquement** (à la compilation).
Objet	Relations entre objets (**composition**), **dynamiques** (modifiables à l'exécution).

Portée \ Objectif	Création (5)	Structure (7)	Comportement (11)
Classe	**Factory Method**	**Adapter** (de classe)	Interpreter, Template Method
Objet	Abstract Factory, Builder, Prototype, **Singleton**	**Adapter** (d'objet), Bridge, **Composite**, Decorator, Façade, Flyweight, Proxy	Chain of Responsibility, **Command**, **Iterator**, Mediator, Memento, **Observer**, State, **Strategy**, Visitor

À connaître par cœur. En gras : patrons vus dans le cours. 5 + 7 + 11 = **23** (Adapter, en version classe et objet, ne compte qu'une fois). Seuls **4 patrons** sont de portée classe : Factory Method, Adapter (classe), Interpreter, Template Method.

MÉTHODE — CLASSER UN PATRON AU DS

Objectif : il sert à *créer* des objets → création ; à *assembler* classes et objets en une structure plus grande → structure ; à *répartir* responsabilités, algorithmes et communication → comportement. **Portée** : la solution repose sur l'**héritage** (figé à la compilation) → classe ; sur des **objets** qui se référencent (composition, délégation, modifiable à l'exécution) → objet. Justifier avec le vocabulaire du cours (« abstraire l'instanciation », « composer », « attribution des responsabilités »).

Lire l'UML des patrons

Chaque patron du cours est présenté par un diagramme de classes, repris en miniature sur les diapositives de code : il faut savoir passer de l'UML au Java et inversement.

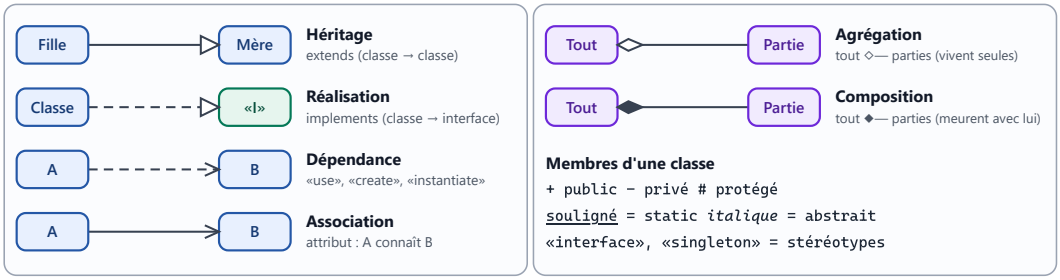


Figure 1.2 — Notation UML utilisée dans le chapitre.

- Trait plein + triangle creux → `extends`
- Pointillé + triangle creux → `implements`
- Pointillé «use» → paramètre, variable locale ou `new`
- Association → attribut (`private StockTrade stock`)
- ◇ agrégation → collection de parties indépendantes
- ◆ composition → les parties appartiennent au tout
- Souligné → `static` ; italique → `abstract`
- « - » privé, « + » public, « # » protégé

1.2 Patrons de création · Singleton

DÉFINITION (COURS) — PATRONS DE CRÉATION

Ils permettent d'**abstraire le processus d'instanciation** et de rendre un système **indépendant** de la manière dont ses objets sont créés, composés et représentés. Création de **classes** : l'**héritage** fait varier la classe instanciée. Création d'**objets** : l'instanciation est **déléguée** à un autre objet.

DÉFINITION (COURS) — SINGLETON

Garantit qu'une **seule instance** d'une classe est créée et fournit un point d'accès **global** à l'objet.

Exemples : gestionnaire de configuration (`ConfigurationManager`), de journalisation (`Logger`).

Création **Portée objet**

Problème concret. Si chaque classe fait `new Logger()` , plusieurs objets écrivent dans le même fichier journal : messages mélangés, compteurs incohérents. Il faut *un seul* `Logger` , partagé par tout le code.

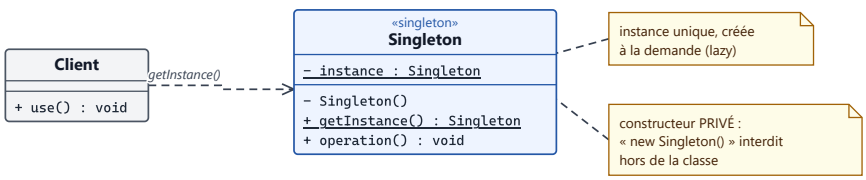


Figure 1.3 — UML du Singleton (souligné = `static` ; constructeur privé).

CODE DU COURS : LES 3 INGRÉDIENTS

```
class Singleton {
  // ① l'instance unique, partagée (static)
  private static Singleton instance;

  // ② constructeur PRIVÉ : new interdit dehors
  private Singleton() { ... }

  // ③ point d'accès global, thread-safe
  public static synchronized Singleton
      getInstance() {
    if (instance == null) {      // 1er appel
      instance = new Singleton(); // lazy
    }
    return instance;  // toujours le même objet
  }
  public void doSomething() { ... }
}
// Logger a = Logger.getInstance();
// Logger b = Logger.getInstance();
// a == b  → true (même objet)
// new Logger() → erreur de compilation
```

À SAVOIR EXPLIQUER (ANNOTATIONS DU COURS)

- `static` : la méthode appartient **à la classe**, pas à une instance. On l'appelle **sans créer d'objet** : `Singleton.getInstance()`. Indispensable, car le constructeur est privé : sans `static`, il faudrait déjà un objet pour obtenir l'objet.
- `synchronized` : verrouille la méthode pour qu'**un seul thread** l'exécute à la fois ; l'autre **attend**. On évite la création de deux instances (*thread-safe*).
- `private` **(constructeur)** : la seule façon d'obtenir l'objet est de passer par `getInstance()`.

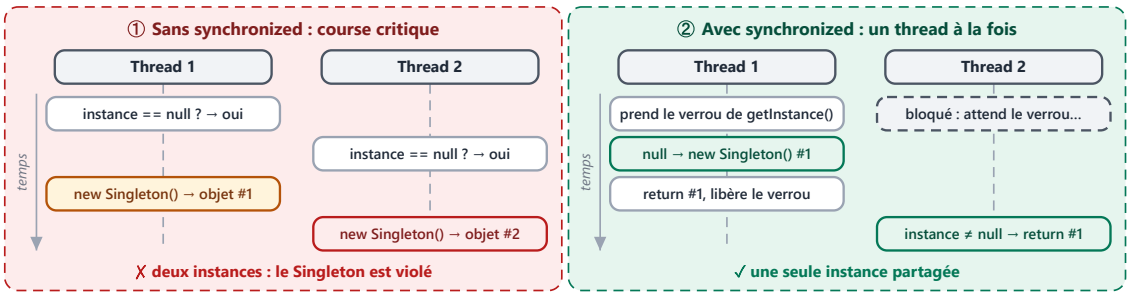


Figure 1.4 — Pourquoi `synchronized` : sans verrou, deux threads peuvent passer le test `instance == null` en même temps et créer deux objets.

Quand l'utiliser ? (cours)

- Il faut s'assurer qu'**une seule instance** d'une classe est créée, **disponible à travers tout le code**.
- En multi-thread, quand **plusieurs threads accèdent aux mêmes ressources** via un même objet.
- Usages : *Logger*, configuration, ressource partagée ; d'autres patrons réalisés en Singleton : **Factories, Abstract Factories, Builder, Prototype**.

Conséquences : + unicité garantie par le langage, accès contrôlé, création paresseuse. − état global, couplage, tests difficiles, coût de `synchronized` **Complément**.

PIÈGES / CORRECTIONS DES DIAPOSITIVES

- Un diagramme note `+Singleton()` et `getInstance() : Singleton()` : **faux**. Le constructeur est `- Singleton()` (privé), le retour est `Singleton`.
- Le client **utilise** le Singleton (dépendance via `getInstance()`) : pas de triangle d'héritage.
- Oublier `static` casse le patron ; oublier `synchronized` le rend dangereux en multi-thread.
- Un Singleton est un vrai **objet** (il peut implémenter une interface), pas une classe « tout `static` ».

1.3 Fabrique (Factory)

DÉFINITION (COURS)

- **Factory Method (GoF)** : définit une interface pour créer un objet, mais laisse les **sous-classes décider** quelle classe instancier.
- Crée des objets **sans exposer la logique d'instanciation** au client, et fait référence à l'objet créé via une **interface commune**.

Intérêt : découpler le client des classes concrètes ; ajouter facilement de nouvelles variantes (ex. `ShapeFactory` qui crée des `Circle`, `Rectangle` ...). **Création** **Portée classe (Factory Method)**

Problème concret. Si le client écrit `new Circle()`, `new Rectangle()` ... partout, il dépend de toutes les classes concrètes et ajouter `Triangle` oblige à tout modifier. La fabrique **regroupe la création en un seul lieu** ; le client ne manipule que l'interface `Shape`.

FABRIQUE SIMPLE : CODE DE LA DIAPOSITIVE

```
public class ProductFactory {
  public Product createProduct(String productID) {
    if (productID == "ID1") return new OneProduct();
    if (productID == "ID2") return new AnotherProduct();
    ...
    return null;
  }
}
```

PIÈGE

`==` compare des **références**, pas le contenu des chaînes : écrire `"ID1".equals(productID)`. Renvoyer `null` provoque plus tard un `NullPointerException` : mieux vaut lever une exception claire.

À RETENIR (RÉPÉTÉ 3 FOIS DANS LE COURS)

Factory → indique comment créer les objets. Singleton → garantit qu'il existe une seule instance de la Factory. Une fabrique sans état propre peut être partagée : on la rend Singleton.

CORRIGÉ : SHAPEFACTORY DU COURS (FABRIQUE + SINGLETON)

```
public interface Shape { void draw(); }
public class Circle implements Shape {
  public void draw() {
    System.out.println("Je dessine un cercle");
  }
}   // Rectangle, Square : même principe

public class ShapeFactory {
  private static ShapeFactory instance;
  private ShapeFactory() { }   // Singleton
  public static synchronized ShapeFactory
      getInstance() {
    if (instance == null)
      instance = new ShapeFactory();
    return instance;
  }
  // Factory : toute la logique de création ici
  public Shape createShape(String type) {
    switch (type.toUpperCase()) { // = equals
      case "CIRCLE":    return new Circle();
      case "RECTANGLE": return new Rectangle();
      case "SQUARE":    return new Square();
      default: throw new IllegalArgumentException(
                 "Forme inconnue : " + type);
    }
  }
}
// Client : jamais de new Circle()
Shape s = ShapeFactory.getInstance()
                      .createShape("circle");
s.draw();   // Je dessine un cercle
```

Factory Method (définition GoF) : la sous-classe décide

La création n'est plus décidée par un `switch` mais par l'**héritage** : `Creator` déclare une méthode de fabrique abstraite, chaque `ConcreteCreator` la redéfinit pour renvoyer un `ConcreteProduct`. D'où la **portée classe**.

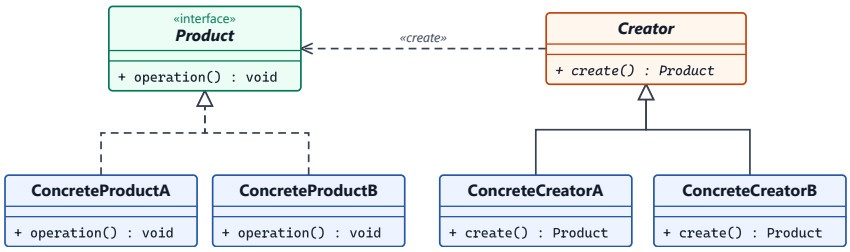


Figure 1.5 — Factory Method : `Creator` déclare `create()`, les créateurs concrets choisissent le produit.

```
abstract class Application { // Creator
  protected abstract Document
      createDocument();  // Factory Method
  public void newDocument() {
    Document d = createDocument();
    d.open();        // code commun
  }
}
class PdfApplication extends Application {
  protected Document createDocument() {
    return new PdfDocument(); // décide
  }
}
```

Variante	Qui choisit la classe concrète ?	Ajouter un produit
Fabrique simple	Un `if` / `switch` sur un paramètre, dans une seule classe	Nouvelle classe + une ligne dans la fabrique ; client inchangé
Factory Method (GoF)	La **sous-classe** du créateur (héritage)	Nouveau produit + nouveau créateur concret, sans modifier l'existant
Abstract Factory Complément	Un objet fabrique qui crée une **famille** de produits cohérents	Nouvelle fabrique concrète

Quand l'utiliser ? (cours)

- Un framework délègue à la fabrique la création d'objets issus d'une **superclasse commune**.
- Besoin de **flexibilité pour ajouter de nouveaux types** d'objets.

Dans l'API Java (cours), sans exposer l'implémentation :

- `DocumentBuilderFactory` → `DocumentBuilder` (XML en DOM) ;
- `SAXParserFactory` → `SAXParser` (XML en SAX) ;

Conséquences : + client découplé des classes concrètes ; + création centralisée, nouvelles variantes faciles. − une classe de plus ; − fabrique simple : le `switch` est à modifier à chaque nouveau type.

- `url.openConnection()` → la bonne `URLConnection` (HTTP, FTP, JAR…) selon le protocole.

EXEMPLE CORRIGÉ — AJOUTER TRIANGLE

(1) `class Triangle implements Shape` avec `draw()` ; (2) `case "TRIANGLE": return new Triangle();` dans `createShape`. Le **client n'est pas modifié**. Avec Factory Method : une sous-classe `TriangleCreator`, sans toucher à l'existant.

Piège : la diapositive écrit `URLConnexion` ; la classe est `java.net.URLConnection`. Ne pas confondre Factory (*comment* créer) et Singleton (*combien* d'instances).

1.4 Patrons structurels · Adaptateur (Adapter)

DÉFINITION (COURS) — PATRONS STRUCTURELS

Ils s'intéressent à la façon dont **classes et objets sont composés pour former des structures plus grandes**.

Modèles de classes : l'**héritage** compose interfaces ou implémentations (statique, ex. Adapter de classe).
Modèles d'objets : on compose des objets ; souplesse de **modifier la composition à l'exécution**. Exemples : Adapter, Proxy, Bridge, Composite.

DÉFINITION (COURS) — ADAPTATEUR

Convertit l'interface d'une classe en une autre interface que les clients attendent. Permet à des classes de **travailler ensemble**, ce qui serait impossible autrement à cause d'**interfaces incompatibles**.

Structure **Portée classe ou objet** · Image : l'adaptateur de prise électrique.

Problème concret (cours). `AudioPlayer` lit les MP3 via `MediaPlayer.play(audioType, filename)`. Une bibliothèque existante lit VLC et MP4 avec une **autre interface**, `AdvancedMediaPlayer` (`playVLC`, `playMP4`). Pour la réutiliser **sans la modifier**, on écrit `MediaAdapter`, qui « parle » `MediaPlayer` à l'extérieur et `AdvancedMediaPlayer` à l'intérieur.

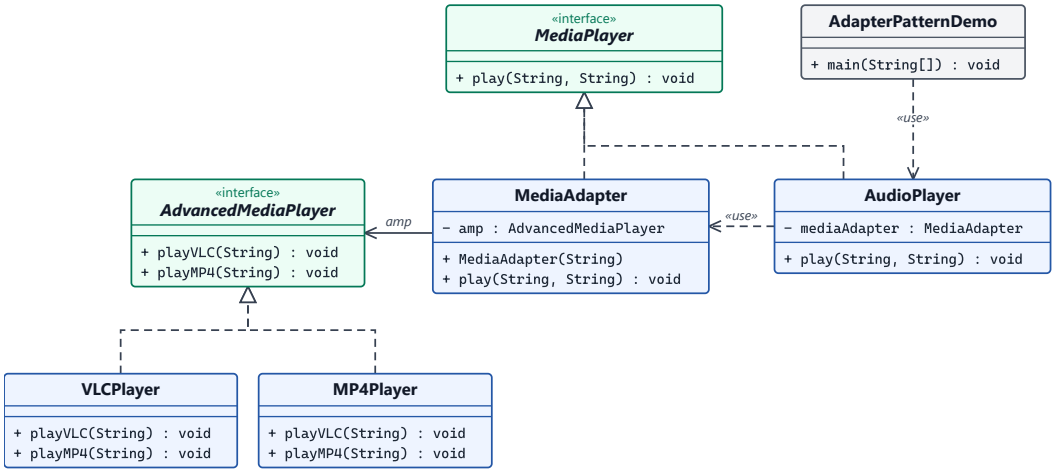


Figure 1.6 — UML de l'exemple du cours : le lecteur multimédia (adaptateur d'objet).

Rôle	Classe du cours	Responsabilité
Target (cible)	`MediaPlayer`	Interface que le client attend : `play(type, fichier)`
Adaptee (adapté)	`AdvancedMediaPlayer`, `VLCPlayer`, `MP4Player`	Code existant, interface incompatible : `playVLC`, `playMP4`
Adapter	`MediaAdapter`	Implémente `MediaPlayer`, contient un `AdvancedMediaPlayer amp` et **traduit** `play()` en `playVLC()` / `playMP4()`
Client	`AudioPlayer` (+ `AdapterPatternDemo`)	Lit le MP3 lui-même, délègue VLC / MP4 à l'adaptateur

```java
public interface MediaPlayer {          // CIBLE
  void play(String audioType, String filename);
}
public interface AdvancedMediaPlayer {  // ADAPTÉE
  void playVLC(String filename);
  void playMP4(String filename);
}
// VLCPlayer : playVLC affiche "Playing VLC file: ..."
// MP4Player : playMP4 affiche "Playing MP4 file: ..."
//             (l'autre méthode : // Do Nothing)

public class MediaAdapter implements MediaPlayer {
  AdvancedMediaPlayer amp;              // ADAPTATEUR
  public MediaAdapter(String audioType) {
    if (audioType.equalsIgnoreCase("vlc"))
      amp = new VLCPlayer();
    else if (audioType.equalsIgnoreCase("mp4"))
      amp = new MP4Player();
  }
  public void play(String audioType, String f) {
    if (audioType.equalsIgnoreCase("vlc"))
      amp.playVLC(f);        // traduction d'appel
    else if (audioType.equalsIgnoreCase("mp4"))
      amp.playMP4(f);
  }
}
```

```java
public class AudioPlayer implements MediaPlayer {
  MediaAdapter mediaAdapter;            // CLIENT
  public void play(String audioType, String f) {
    if (audioType.equalsIgnoreCase("mp3"))   // natif
      System.out.println("Playing MP3 file: " + f);
    else if (audioType.equalsIgnoreCase("vlc")
          || audioType.equalsIgnoreCase("mp4")) {
      mediaAdapter = new MediaAdapter(audioType);
      mediaAdapter.play(audioType, f);
    } else
      System.out.println("Invalid Media format: "
                         + audioType);
  }
}
// AdapterPatternDemo.main :
AudioPlayer p = new AudioPlayer();
p.play("mp3", "beyond the horizon.mp3");
p.play("mp4", "alone.mp4");
p.play("vlc", "far far away.mp3");
p.play("avi", "mind me.mp3");
```

SORTIE CONSOLE

```
Playing MP3 file: beyond the horizon.mp3
Playing MP4 file: alone.mp4
Playing VLC file: far far away.mp3
Invalid Media format: avi
```

Séquence du cours : MP3 → `AudioPlayer` lit seul ; MP4 → `AudioPlayer.play → MediaAdapter.play → MP4Player.playMP4`.

Adaptateur d'objet ou de classe ? L'exemple du cours est un adaptateur d'**objet** : `MediaAdapter` *contient* un `AdvancedMediaPlayer` (composition, choisi à l'exécution). Un adaptateur de **classe** *hérite* de l'adapté (statique) : `class MP4ClassAdapter extends MP4Player implements MediaPlayer`, dont `play()` appelle `playMP4(f)`.

Quand l'utiliser : réutiliser une classe existante (bibliothèque, code ancien, service externe) dont l'interface ne convient pas.
Conséquences : + client et adapté inchangés, conversion isolée ; – une indirection de plus ; chaque nouveau format modifie `MediaAdapter` et `AudioPlayer`.

PIÈGES / CORRECTIONS DES DIAPOSITIVES

- C'est le **type passé en paramètre**, pas l'extension, qui décide : `play("vlc", "far far away.mp3")` affiche *Playing VLC file.*
- Coquilles : « MP4/VMC … AdvancerMediaPlayer » = MP4/VLC et `AdvancedMediaPlayer` ; la séquence MP3 « détecte "mp4" » doit se lire « "mp3" » ; le `MP3Player.playMP3()` de la séquence n'existe pas (c'est `AudioPlayer` qui lit le MP3).
- L'UML note `+play()` sans paramètres ; le code a `play(String audioType, String filename)`.

1.5 Composite

DÉFINITION (COURS)

Compose des objets en arborescence pour représenter des hiérarchies partielles ou complètes (parties / tout). Permet aux clients de **traiter de manière uniforme** les objets individuels et les compositions d'objets. **Structure** **Portée objet**

Problème concret (cours). Un éditeur graphique manipule lignes, rectangles et *groupes* de formes (eux-mêmes imbriqués) ; le client appelle `renderShapeToScreen()` sur tout élément, sans `instanceof`. **Idée clé** : le composite **implémente** l'interface commune **et contient** des éléments de cette interface → un arbre.

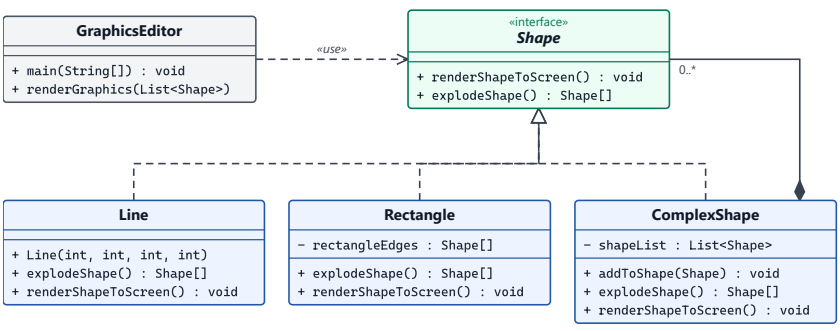


Figure 1.7 — UML du cours (◆ = composition récursive).

Rôle	Classe du cours · responsabilité
Component	`Shape` (interface) : `renderShapeToScreen()`, `explodeShape()`
Leaf	`Line` : s'affiche, ne se décompose qu'en elle-même
Composite	`ComplexShape` (et `Rectangle` = 4 arêtes) : contient des `Shape`, propage les appels
Client	`GraphicsEditor` : n'utilise que `Shape` («use»)

CODE DU COURS (CORRIGÉ ✓)

```
public interface Shape {              // COMPONENT
  void renderShapeToScreen();
  Shape[] explodeShape();
}
public class Line implements Shape { // FEUILLE
  public Line(int x1, int y1, int x2, int y2) { }
  public Shape[] explodeShape() {
    return new Shape[] { this };      // ✓
  }
  public void renderShapeToScreen() {
    System.out.println("Ligne affichée");
  }
}
public class ComplexShape implements Shape {
  private List<Shape> shapeList =   // ✓ typée
      new ArrayList<>();            // COMPOSITE
  public void addToShape(Shape s) {
    shapeList.add(s);
  }
  public Shape[] explodeShape() {    // ✓
    return shapeList.toArray(new Shape[0]);
  }
  public void renderShapeToScreen() {
    for (Shape s : shapeList)
      s.renderShapeToScreen(); // selon le type réel
  }
}
// Rectangle : tableau de 4 Line (ses arêtes),
// renderShapeToScreen() les affiche une par une.
```

```
public class GraphicsEditor {          // CLIENT
  public static void main(String[] args) {
    ComplexShape toit = new ComplexShape();
    toit.addToShape(new Line(0, 10, 5, 15));
    toit.addToShape(new Line(5, 15, 10, 10));
    ComplexShape dessin = new ComplexShape();
    dessin.addToShape(toit); // composite ⊂ composite
    dessin.addToShape(new Rectangle(0, 0, 10, 10));
    dessin.addToShape(new Line(8, 12, 8, 18));
    dessin.renderShapeToScreen(); // aucun instanceof
  }
}
```

EXEMPLE CORRIGÉ — COMBIEN DE LIGNES AFFICHÉES ?

`dessin.renderShapeToScreen()` descend récursivement : `toit` → 2 lignes, `Rectangle` → 4 arêtes, antenne → 1. Total : **7 « Ligne affichée »** pour **un seul** appel sur la racine : c'est le traitement uniforme promis par le patron.

Quand l'utiliser : données **arborescentes** (dessin, menus / sous-menus, fichiers / dossiers, organigramme, panneau contenant des boutons) ; le client doit ignorer la différence simple / groupe. **Conséquences** : + client simple, nouveaux types faciles, profondeur quelconque ; – difficile d'interdire certains enfants ; la gestion des enfants n'a pas de sens pour une feuille.

PIÈGES / CORRECTIONS DES DIAPOSITIVES

- `Shape[] shapeParts = this;` ne compile pas → `return new Shape[] { this };`
- `(Shape[]) shapeList.toArray()` lève une `ClassCastException` (`toArray()` renvoie un `Object[]`) → `toArray(new Shape[0])`.
- Avec une `List` brute, `for (Shape s : shapeList)` ne compile pas → `List<Shape>`.
- UML : `ComplexShape` est relié deux fois à `Shape`, par une **réalisation** (il est un `Shape`) et une **composition** (il contient des `Shape`).

1.6 Patrons comportementaux · Itérateur, Stratégie

DÉFINITION (COURS)

Les patrons comportementaux concernent les **algorithmes** et l'**attribution des responsabilités entre les objets** (choisir le type de l'objet au moment de l'exécution). **Modèles de classe** : l'**héritage** répartit le comportement (Template Method, Interpreter). **Modèles d'objets** : la **composition plutôt que l'héritage** ; des **objets pairs coopèrent** pour une tâche qu'aucun ne peut réaliser seul. Exemples : Commande, Iterator, Observer, Strategy.

À retenir : chaque famille (création, structure, comportement) oppose une variante « classe / héritage / statique » à une variante « objet / composition / dynamique ».

Itérateur (Iterator) : parcourir les éléments d'une collection **sans exposer sa représentation interne** (tableau, liste, arbre…). L'agrégat crée son propre itérateur (`createIterator()`) ; en Java : `it.hasNext()` / `it.next()`, utilisés en cachette par `for (x : liste)`.

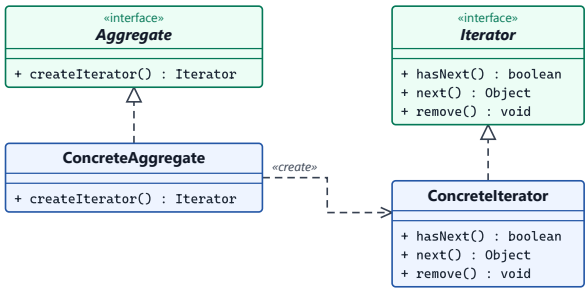


Figure 1.8 — UML de l'Iterator (cours).

Stratégie (Strategy) : une famille d'**algorithmes interchangeables**. Le `Context` garde une référence vers une `Strategy` et peut la **changer à l'exécution** (`setStrategy`).

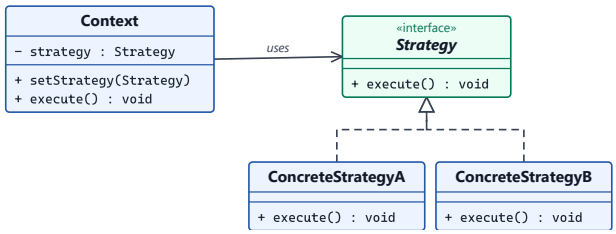


Figure 1.9 — UML de la Strategy (cours).

```
interface Remise { double appliquer(double p); }
class Panier {                          // Context
  private Remise strategy = p -> p; // aucune
  void setStrategy(Remise r) { strategy = r; }
  double total(double prix) {
    return strategy.appliquer(prix); // délègue
  }
}  // setStrategy(p -> p * 0.7) ; total(100) → 70.0
```

1.7 Commande (Command)

DÉFINITION (COURS)

Encapsule une demande dans un objet. Permet de **paramétrer les clients** ayant des demandes différentes et d'**enregistrer la demande dans une file d'attente**. Comportement Portée objet

Problème concret (cours : la bourse). Un `Agent` reçoit des ordres d'achat et de vente, les met en file et les exécute, sans connaître le détail de chaque opération (`buy()` , `sell()` de `StockTrade`). Chaque demande devient un **objet** `Order` doté d'une seule méthode, `execute()` .

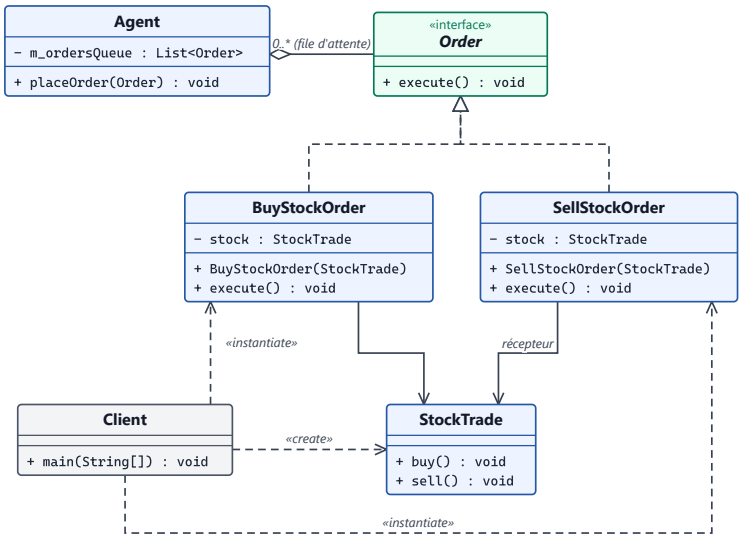


Figure 1.10 — UML de l'exemple du cours (ordres de bourse).

Rôle	Classe du cours · responsabilité
Command	`Order` : déclare `execute()`
ConcreteCommand	`BuyStockOrder` , `SellStockOrder` : mémorisent le récepteur, appellent `buy()` / `sell()`
Receiver	`StockTrade` : fait réellement le travail
Invoker	`Agent` : met les ordres en **file** et les déclenche
Client	`Client` : crée récepteur et commandes («instantiate»), les confie à l'agent

```
public interface Order { void execute(); } // COMMAND

class StockTrade {                          // RECEIVER
  public void buy()  {
    System.out.println("You want to buy stocks"); }
  public void sell() {
    System.out.println("You want to sell stocks"); }
}
class BuyStockOrder implements Order {      // CONCRETE
  private StockTrade stock;
  public BuyStockOrder(StockTrade st) { stock = st; }
  public void execute() { stock.buy(); }   // délègue
}
// SellStockOrder : identique, execute() → stock.sell()

public class Client {                       // CLIENT
  public static void main(String[] args) {
    StockTrade stock = new StockTrade();
    BuyStockOrder  bso = new BuyStockOrder(stock);
    SellStockOrder sso = new SellStockOrder(stock);
    Agent agent = new Agent();
    agent.placeOrder(bso);
    agent.placeOrder(sso);
  }
}
```

AGENT CORRIGÉ (INVOKER)

```
class Agent {
  private Queue<Order> ordersQueue =
      new LinkedList<>();
  void placeOrder(Order order) {
    ordersQueue.add(order);           // enregistre
    Order next = ordersQueue.poll(); // 1re retirée
    next.execute();  // sans connaître le type
  }
}
```

SORTIE CONSOLE

```
You want to buy stocks
You want to sell stocks
```

Séquence du cours : (1-4) le client crée `stock` , `bso` , `sso` , `agent` ; (5) `placeOrder(bso)` : ajout en file puis retrait ; (6) `execute()` ; (7) `stock.buy()` ; (8-10) idem avec `sso` et `sell()` . Seul l'objet commande connaît le type d'ordre.

PIÈGES / CORRECTIONS

- Diapositive : `List m_ordersQueue … addLast(order); order.execute(m_ordersQueue.getFirstAndRemove())` ne compile pas (`List` n'a ni `addLast` ni `getFirstAndRemove` , `execute()` sans paramètre, `;` manquant) → version corrigée ci-dessus.
- Coquilles UML : `StrockTrade` , `BuyStockOrder(stock : StockOrder)` = `StockTrade` .
- **Invoker** (Agent, déclenche) ≠ **Receiver** (StockTrade, fait le travail).

Quand l'utiliser (cours) : mettre des demandes en **file d'attente**, les différer, les planifier ; paramétrer un objet (bouton, menu, agent) par l'action à exécuter ; Complément historique, **annuler / refaire** (`undo()`). **Conséquences** : + découple l'**émetteur** (Agent) du **récepteur** (StockTrade) ; + nouvelle commande = nouvelle classe, Agent inchangé ; + une demande devient un objet (stockée, transmise, rejouée) ; − beaucoup de petites classes.

1.8 Observateur (Observer)

DÉFINITION (COURS) — À CONNAÎTRE PAR CŒUR

Définit une dépendance de un à plusieurs objets de sorte que lorsqu'un objet change d'état, toutes ses dépendances sont notifiées et mises à jour automatiquement. → ① le sujet change d'état, ② il notifie tous ses observateurs, ③ chacun se met à jour. Comportement Portée objet

Problème concret (cours). Un entier (l'état du `Subject`) doit être affiché en binaire, en octal et en hexadécimal ; à chaque changement, **tous** les affichages se mettent à jour **automatiquement**. Le sujet ne connaît que l'abstraction `Observer` , pas les classes d'affichage.

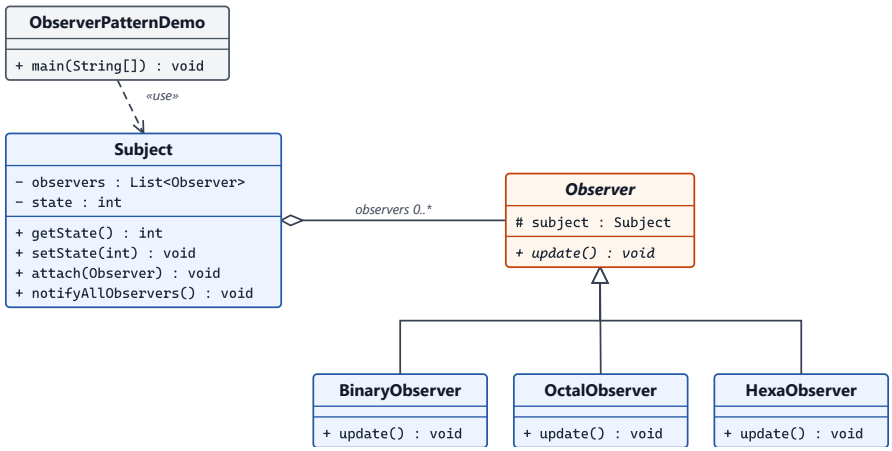


Figure 1.11 — UML de l'exemple du cours : le sujet agrège 0.. observateurs (conversion binaire / octal / hexadécimal).*

Rôle	Classe du cours	Responsabilité
Subject (sujet)	`Subject`	Garde l'état et la liste des observateurs ; `attach()` ; `setState()` déclenche `notifyAllObservers()`
Observer	`Observer` (classe abstraite)	Déclare `update()` ; garde `protected Subject subject`
ConcreteObserver	`BinaryObserver` , `OctalObserver` , `HexaObserver`	S'attachent au sujet dans leur constructeur ; `update()` lit `subject.getState()` et affiche
Client	`ObserverPatternDemo`	Crée le sujet et les observateurs, puis change l'état

```java
public class Subject {
  private List<Observer> observers =
      new ArrayList<Observer>();
  private int state;
  public int getState() { return state; }
  public void setState(int state) {
    this.state = state;
    notifyAllObservers();  // changement ⇒ notif.
  }
  public void attach(Observer o) {
    observers.add(o);      // abonnement
  }
  public void notifyAllObservers() {
    for (Observer o : observers)
      o.update();          // diffusion à tous
  }
}
public abstract class Observer {
  protected Subject subject;
  public abstract void update();
}
```

```java
public class BinaryObserver extends Observer {
  public BinaryObserver(Subject subject) {
    this.subject = subject;
    this.subject.attach(this); // s'abonne seul
  }
  public void update() {
    System.out.println("Binary String: " +
      Integer.toBinaryString(subject.getState()));
  }
} // Octal / Hexa : idem avec toOctalString,
  // toHexString(...).toUpperCase()

// ObserverPatternDemo.main :
Subject subject = new Subject();
new HexaObserver(subject);    // attaché en 1er
new OctalObserver(subject);   // en 2e
new BinaryObserver(subject);  // en 3e
System.out.println("First state change: 15");
subject.setState(15);
System.out.println("Second state change: 10");
subject.setState(10);
```

SORTIE CONSOLE (DIAPOSITIVE)

```
First state change: 15
Hex String: F
Octal String: 17
Binary String: 1111
Second state change: 10
Hex String: A
Octal String: 12
Binary String: 1010
```

Hex → Octal → Binary = **ordre d'attachement** (parcours de la liste). 15 = F = 17_8 = 1111_2 ; 10 = A = 12_8 = 1010_2.

- C'est `setState()` qui déclenche la notification : le client n'appelle jamais `update()`.
- Modèle **pull** : l'observateur va chercher la valeur (`getState()`) ; en **push**, elle serait passée en paramètre **Complément**.
- **Conséquences** : + couplage faible (le sujet ne connaît que `Observer`), nouvel observateur sans modifier `Subject`, diffusion 1 → N ; − ordre implicite, cascades possibles, pas de `detach`.
- Base des **architectures réactives et événementielles** (publication / abonnement, ch. 3).

PIÈGES / CORRECTIONS

- L'UML dit `notifyAll()`, le code `notifyAllObservers()` : garde ce dernier, car `notifyAll()` est une méthode `final` de `Object` (threads).
- Ici `Observer` est une **classe abstraite** (`extends`) ; dans le schéma générique, souvent une interface.

1.9 Synthèse et à retenir pour le DS

Patron	Famille · portée	Intention (cours)	Mot-clé déclencheur	Exemple du cours
Singleton	**Création** objet	Une seule instance + accès global	« unique », « partagé », « global »	Logger, configuration
Factory	**Création** classe	Créer sans exposer la logique d'instanciation ; les sous-classes décident	« créer selon un type », « sans connaître la classe »	`ShapeFactory`, parseurs XML
Adapter	**Structure** classe/objet	Convertir une interface en celle attendue	« incompatible », « existant », « réutiliser »	`MediaAdapter`
Composite	**Structure** objet	Arborescence ; traiter uniformément simple et composé	« hiérarchie », « contient », « groupe »	`ComplexShape`
Command	**Comportement** objet	Encapsuler une demande dans un objet, la mettre en file	« file », « ordre », « annuler », « historique »	`BuyStockOrder`, `Agent`
Observer	**Comportement** objet	Dépendance 1 → N, notification automatique	« prévenir », « abonnés », « dès que … change »	`Subject`, `BinaryObserver`
Iterator	**Comportement** objet	Parcourir sans exposer la structure	« parcourir », « élément suivant »	mini-UML
Strategy	**Comportement** objet	Algorithmes interchangeables à l'exécution	« plusieurs façons de calculer »	mini-UML

MÉTHODE — CHOISIR LE PATRON DANS UN ÉNONCÉ

1. Souligner le **besoin** : créer ? assembler ? faire communiquer ?
2. Repérer les mots-clés du tableau ci-dessus.
3. Nommer les **rôles** avec les classes de l'énoncé (Target/Adapter, Subject/Observer…).
4. Justifier par la définition, puis donner une conséquence (+ et −).

COUPLES SOUVENT CONFONDUS

- **Factory / Singleton** : *comment* créer ≠ *combien* d'instances (combinables).
- **Adapter / Composite** : changer une interface ≠ bâtir un arbre de même interface.
- **Command / Strategy** : demande à exécuter (souvent plus tard, en file) ≠ façon de calculer choisie par un contexte.
- **Observer / Command** : diffuser un changement à N abonnés ≠ transmettre une action à un récepteur.

LIEN AVEC LE CHAPITRE 2 (SOLID)

Les patrons **appliquent** les principes : Fabrique et Observer dépendent d'**abstractions** (DIP) ; Commande, Observer, Strategy, Adaptateur permettent d'**ajouter sans modifier** (OCP) ; Commande = une classe par action (SRP) ; feuilles et composites substituables à `Shape` (LSP).

SAVOIR-FAIRE ATTENDUS

- ☐ Écrire un Singleton thread-safe ; expliquer `private`, `static`, `synchronized`.
- ☐ Écrire une fabrique (`equals` / `switch`) ; dire quoi modifier pour un nouveau produit.
- ☐ Réciter « Factory → comment créer ; Singleton → une seule instance de la Factory ».
- ☐ Associer chaque classe d'un UML à son rôle (Target, Adaptee, Adapter ; Component, Leaf, Composite ; Command, Receiver, Invoker ; Subject, Observer).
- ☐ Prédire la sortie des démos Adapter, Command et Observer (ordre d'attachement !).
- ☐ Traduire les flèches UML en Java ; remplir le tableau des 23 patrons.

PIÈGES CLASSIQUES

- Constructeur du Singleton **privé** (pas `+Singleton()`).
- `equals` et non `==` pour comparer des `String`.
- `URLConnection` (pas « URLConnexion ») ; `notifyAllObservers()` (pas `notifyAll()`).
- Grady Booch = préfacier, pas un auteur GoF.
- `toArray(new Shape[0])`, pas un cast de `toArray()`.
- Seuls 4 patrons sont de portée classe.

1.10 Exercices corrigés

Le cours ne contient pas d'exercices pour ce chapitre : les corrigés ci-dessous sont des propositions construites sur les exemples du cours.
Proposition de corrigé — non fournie dans le cours

Exercice 1 — Classification. Donner l'objectif et la portée de : Builder, Template Method, Proxy, Observer, Factory Method, Façade, Memento, Adapter (version héritage).

CORRIGÉ

Builder : création / objet. Template Method : comportement / **classe**. Proxy : structure / objet. Observer : comportement / objet. Factory Method : création / **classe**. Façade : structure / objet. Memento : comportement / objet. Adapter par héritage : structure / **classe**. Portée classe = la solution repose sur l'héritage, fixé à la compilation.

Exercice 2 — Trouver les erreurs. Ce code est censé être un Singleton thread-safe.

```
public class Config {
    private Config instance;
    public Config() { }
    public synchronized Config getInstance() {
        if (instance == null) instance = new Config();
        return instance;
    }
}
```

CORRIGÉ

(1) `instance` doit être `static`, sinon chaque objet a la sienne. (2) Le constructeur doit être `private`, sinon `new Config()` reste possible. (3) `getInstance()` doit être `static`, sinon il faut déjà un objet pour l'appeler. Corrigé : `private static Config instance; private Config() { } public static synchronized Config getInstance() { … }`. `synchronized` est bien présent.

Exercice 3 — Fabrique. (a) Écrire une fabrique `VehiculeFactory` qui crée une `Voiture` ou une `Moto` (interface `Vehicule`). (b) Que modifier pour ajouter `Camion` ? (c) Rendre la fabrique unique et dire l'intérêt de la combinaison.

CORRIGÉ

```
public class VehiculeFactory {
    public Vehicule creer(String type) {
        if ("VOITURE".equalsIgnoreCase(type)) return new Voiture();
        if ("MOTO".equalsIgnoreCase(type))    return new Moto();
        throw new IllegalArgumentException("Type inconnu : " + type);
    }
}
```

(b) Créer `Camion implements Vehicule` et ajouter une ligne dans `creer` ; les clients, qui ne manipulent que `Vehicule`, ne changent pas. (c) `private static VehiculeFactory instance;`, constructeur `private`, `public static synchronized VehiculeFactory getInstance()` ; usage `VehiculeFactory.getInstance().creer("MOTO")`. Intérêt (cours) : la **Factory indique comment créer**, le **Singleton garantit une seule instance de la Factory**.

Exercice 4 — Prédire les sorties. (a) Adaptateur : `audioPlayer.play("MP4", "a.mp4"); audioPlayer.play("vlc", "b.avi"); audioPlayer.play("wav", "c.wav");` (b) Observer : `Subject s = new Subject(); new BinaryObserver(s); new HexaObserver(s); s.setState(255); s.setState(8);` (c) Comment ajouter un `DecimalObserver` ?

CORRIGÉ

(a) `Playing MP4 file: a.mp4` (`equalsIgnoreCase`) ; `Playing VLC file: b.avi` (le type décide, pas l'extension) ; `Invalid Media format: wav`. (b) `Binary String: 11111111`, `Hex String: FF`, `Binary String: 1000`, `Hex String: 8` : Binary d'abord car attaché en premier. (c) `class DecimalObserver extends Observer` : constructeur `this.subject = subject; this.subject.attach(this);`, `update()` affiche `subject.getState()`. `Subject` **n'est pas modifié** : c'est l'intérêt du patron.

Exercice 5 — Quel patron ? (a) Une seule connexion à la base partagée par toute l'application. (b) Des menus contenant des sous-menus et des éléments. (c) Une API de paiement externe dont les méthodes ne correspondent pas à notre interface `Paiement`. (d) Mettre à jour un tableau de bord dès qu'un capteur publie une mesure. (e) Créer un document PDF, Word ou HTML selon un paramètre, sans que le client connaisse les classes. (f) Enregistrer les actions d'un éditeur pour pouvoir les annuler. (g) Choisir à l'exécution entre plusieurs algorithmes de tri.

CORRIGÉ

(a) Singleton. (b) Composite. (c) Adaptateur. (d) Observer (« dès que »). (e) Fabrique (éventuellement Singleton pour la fabrique). (f) Commande (objets avec `undo()`, historique). (g) Strategy.

CHAPITRE 2

Principes de conception orientée objet

« Règles de conception logicielle OO — Les principes d'or : SOLID, cohésion & couplage ». Question centrale : *pourquoi le code « pourrit »-il, et quelles règles l'en empêchent ?*

Objectifs

- Reconnaître les **7 risques** de conception (symptômes du code qui pourrit).
- Énoncer et appliquer les **5 principes SOLID** (niveau classes).
- Appliquer les **6 principes de packages** : cohésion (REP, CCP, CRP) et couplage (ADP, SDP, SAP).
- Calculer et interpréter les **métriques** C_a, C_e, I, A, D.

Ce qui tombe au DS

- Définition exacte de chaque principe et un exemple de violation.
- Corriger un code ou un UML qui viole SRP/OCP/LSP/ISP/DIP.
- QCM du cours (ISP, DIP, LSP, métriques).
- Calculer I, A, D et situer un package ; casser un cycle (ADP) ; CCP ↔ SRP, CRP ↔ ISP.

Pourquoi des principes ? Les 7 risques de conception

QUESTION D'EXPLORATION N° 1 — RÉPONSE ATTENDUE

« *Qu'est-ce qui rend un logiciel difficile à maintenir après 2 ans de production : la quantité de code ou la structure des dépendances ?* » C'est surtout la **structure des dépendances** : un gros code bien découplé reste maintenable, un petit code où tout dépend de tout casse à chaque évolution. Les principes de ce chapitre sont donc des **règles sur les dépendances** (qui dépend de qui, dans quel sens, du concret ou de l'abstrait).

Deux niveaux. Classes : SOLID (SRP, OCP, LSP, ISP, DIP) réduit le couplage et maximise la **testabilité**. **Packages** : la **cohésion** (REP, CCP, CRP) dit quoi regrouper, le **couplage** (ADP, SDP, SAP) comment relier les packages. Haute cohésion + faible couplage ⇒ **près de −40 % de coûts de maintenance**. *Principe de réalité* (« Perfect is the enemy of good ») : pas de « potion magique » ; les principes sont des **boussoles pragmatiques**, pas des dogmes.

Les 7 risques (Robert C. Martin) sont les **symptômes observables** d'une mauvaise structure : « *les effets observables quand les règles SOLID sont absentes ou mal appliquées* » (slide 13).

Risque	Définition (cours)	Exemple du cours	Remède / principe
Rigidité	Changements difficiles : une modification entraîne un **effet domino** sur de nombreux modules.	Modifier `Produit` oblige à réécrire 15 modules (Facturation, Panier, Stock…).	Découpler : OCP, DIP ; SDP, ADP
Fragilité	Le logiciel casse à des endroits **sans relation directe** avec la modification (couplage caché).	Corriger le bouton de profil désactive le **tunnel de paiement**.	SRP, LSP, dépendances explicites, tests
Immobilité	Incapacité à **réutiliser** des composants dans d'autres projets.	Module **SMS** soudé à la BDD, à la config et aux classes du projet A.	DIP, ISP ; REP, CRP
Viscosité	Les **hacks** sont plus faciles que respecter le design d'origine.	Ajouter une **variable globale** au lieu de changer une signature.	Ne pas interdire le raccourci : **abaisser le coût du bon chemin**
Répétition inutile	Duplication (copier/coller) par manque d'abstraction ; chaque changement métier se propage.	Validation des e-mails dupliquée dans **5 formulaires**.	**DRY** : une fonction partagée (`validerCode()`)
Opacité	Code difficile à lire, comprendre et faire évoluer.	Fonction de **300 lignes**, variables `x` , `temp` , `data1` .	Ne dépend pas du couplage : rapport **effort de lecture / d'écriture**
Complexité inutile	*Over-engineering* : abstractions prématurées non requises.	**Fabriques d'usines génériques** pour un script CSV qui ne changera jamais.	**YAGNI** : construire pour le besoin réel

PIÈGE — NUMÉROTATION INCOHÉRENTE DANS LES SLIDES

La synthèse (p. 5) numérote 5 = Répétition, 6 = Opacité, 7 = Complexité ; les slides de détail : 5 = Complexité (p. 12), 6 = Répétition (p. 10), 7 = Opacité (p. 11). Au DS, citez les risques **par leur nom**. Lecture d'ensemble : les 4 premiers viennent des **dépendances** (SOLID, packages) ; les 3 derniers de la **qualité du code** (DRY, lisibilité, YAGNI). Appliquer SOLID « au cas où » mène à la complexité inutile.

SOLID : vue d'ensemble

Slide 13 : « *Les 5 principes SOLID réduisent le couplage et maximisent la testabilité des classes.* » Les formulations sont à connaître **mot pour mot** ; le reste du tableau sert de grille d'analyse au DS.

Principe	Formulation (slide 13)	Signe de violation	Correction type	Patrons du ch. 1
S — SRP	Une classe ne doit avoir qu'**une seule raison de changer** ; elle isole un concept métier unique.	Classe « couteau suisse »	Découper par responsabilité	Commande
O — OCP	**Ouvert aux extensions, fermé aux modifications** ; on étend par polymorphisme.	`switch` / `if` sur un type	Interface + polymorphisme	Stratégie, Fabrique, Observateur
L — LSP	Les sous-classes doivent être **substituables** à leur classe de base sans altérer le client.	Exception ou méthode vide héritée	Revoir la hiérarchie, interface optionnelle	Composite
I — ISP	Des **interfaces spécifiques à chaque client** plutôt qu'une interface « fourre-tout ».	Méthodes inutiles imposées	Une interface par client	Adaptateur (cible minimale)
D — DIP	*Depend on abstractions, not concretions* ; le haut niveau ne dépend pas du bas niveau.	`new` du concret dans le métier	Interface (port) + injection	Adaptateur, Fabrique

S et O — Responsabilité unique (SRP), Ouvert/fermé (OCP)

S
SRP

DÉFINITION (SLIDE 14)

Une classe ne doit avoir qu'une seule raison de changer, donc une seule responsabilité (une seule fonctionnalité métier).

Symptôme : une classe qui mélange données, règles métier, persistance, affichage ou communication. Changer le SGBD, le serveur SMTP ou une règle d'inscription obligerait à modifier la *même* classe.

MAUVAISE CONCEPTION (SLIDE 14)

```
public class User {
  private String name, email;   // données
  public void register()     { /* règles métier */ }
  public void envoyerEmail() { /* SMTP… */ }
  public void sauvegarder()  { /* INSERT INTO… */ }
}   // 3 raisons de changer
```

BONNE CONCEPTION

PROPOSITION — NON FOURNIE DANS LE COURS

```
class User { String name, email;
             void register() { … } }   // métier
class EmailService {                   // envoi
  void envoyerEmail(User u) { … } }
class UserRepository {                 // persistance
  void sauvegarder(User u) { … } }
```

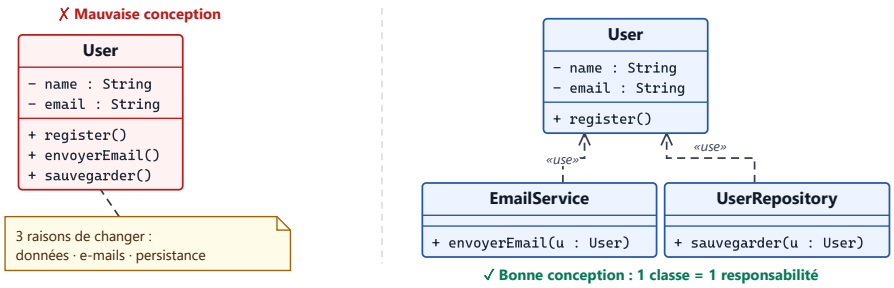


Figure 2.1 — SRP : la classe User « couteau suisse » (gauche) découpée en User, EmailService et UserRepository (droite ; zone masquée sur la slide).

BÉNÉFICE ET PIÈGES

- **Bénéfice (slide 14)** : classe plus simple, plus facile à maintenir et à tester isolément (tester `register()` sans BDD ni serveur mail).
- « Une seule responsabilité » ≠ « une seule méthode » : le critère est **une seule raison de changer**. SRP au niveau package = **CCP** (QCM slide 59).

O
OCP

DÉFINITION (SLIDE 15)

Les entités doivent être ouvertes à l'extension, mais fermées à la modification : ajouter des fonctionnalités sans changer le code existant. **Ouvert** : extensible à un comportement non planifié ; **fermé** : sans modifier le code initial, déjà testé et livré.

Symptôme : un `switch` ou une cascade de `if` sur un type (chaîne, énumération, `instanceof`) : chaque nouveau cas oblige à rouvrir la classe (risque d'erreurs, recompilation).

MAUVAISE CONCEPTION (SLIDE 15)

```
public class Payment {
  public void pay(String pType, double a) {
    switch (pType) {
      case "CREDIT_CARD": /* … */ break;
      case "PAYPAL":      /* … */ break;
      case "CASH":        /* … */ break;
    }  // nouveau moyen ⇒ on MODIFIE Payment
  }
}
```

BONNE CONCEPTION (SLIDE 16)

```
interface PaymentMethod { void pay(); double
getAmount(); }
class CreditCardPayment implements PaymentMethod { … }
class CashPayment       implements PaymentMethod { … }
class PaymentProcessor {
  void pay(PaymentMethod p) { p.pay(); } // liaison
dynamique
}
// EXTENSION : nouvelle classe, rien d'autre ne change
class PayPalPayment implements PaymentMethod { … }
```

Deuxième exemple du cours (slide 25) : `ShapeManipulator` teste le type de chaque forme (non conforme) ; `SmartShapeManipulator` appelle `ShapeInterface.draw()` (conforme) : une nouvelle forme s'ajoute sans modifier une ligne du client.

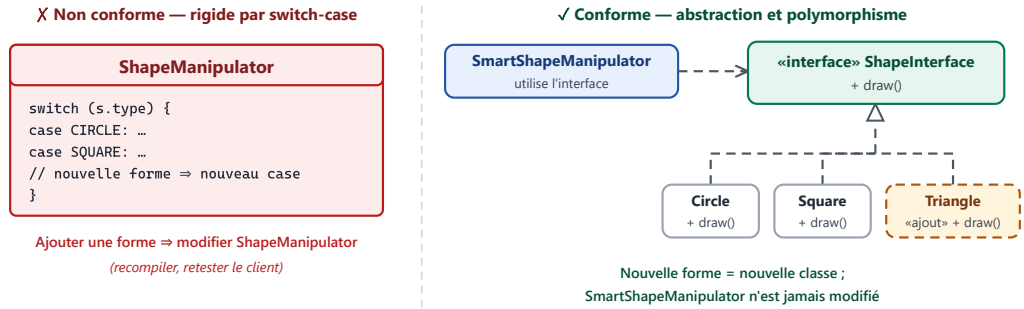


Figure 2.2 — OCP (slide 25) : aiguillage sur le type (non conforme) vs polymorphisme via ShapeInterface (conforme).

MÉTHODE : RENDRE UN CODE CONFORME À L'OCP

1. Repérer ce qui **varie** (le corps de chaque `case`).
2. Créer une **interface** qui déclare l'opération commune.
3. Une **classe par cas**, qui implémente l'interface.
4. Le client manipule le **type abstrait** (polymorphisme) ; la création du bon objet va dans une **fabrique**.

PIÈGES DS

- « Fermé » ≠ « jamais modifié » : on corrige les bugs ; on n'ouvre pas la classe pour **ajouter un comportement**.
- Bénéfice (cours) : code **plus extensible et plus stable**. OCP traite rigidité et viscosité.
- Slide 26 : OCP + LSP sécurisent les évolutions sans impacter le client (plus de `switch`, contrat respecté par toutes les sous-classes).

L et I — Substitution de Liskov (LSP), Ségrégation des interfaces (ISP)

L
LSP

DÉFINITION (SLIDES 13 ET 17)

Les sous-classes doivent pouvoir être utilisées à la place de leurs classes mères sans modifier le comportement du programme (ni le client, ni le contrat de la mère).

Symptôme : une sous-classe qui ne peut pas tenir une promesse de sa mère (`UnsupportedOperationException`, méthode vide, valeur absurde), ou un client obligé de tester `instanceof`.

MAUVAISE CONCEPTION (SLIDE 17)

```
class Oiseau { void voler() { … } }
class Canard extends Oiseau { void voler() { … } }
class Pingouin extends Oiseau {
  void voler() { throw new
    UnsupportedOperationException("ne vole pas"); }
}
void faireVoler(Oiseau o) { o.voler(); } // plante
```

BONNE CONCEPTION (SLIDE 18)

```
class Oiseau { void seDeplacer() { … } }
interface Volant { void voler(); }
class Canard extends Oiseau implements Volant { … }
class Pingouin extends Oiseau {
  void seDeplacer() { /* marche, nage */ } }
void deplacer(Oiseau o) { o.seDeplacer(); } // OK
```

Slide 18 : « **Pingouin et Canard sont substituables à Oiseau**, sans casser `deplacer()` ». Le polymorphisme fonctionne, le code est plus fiable.

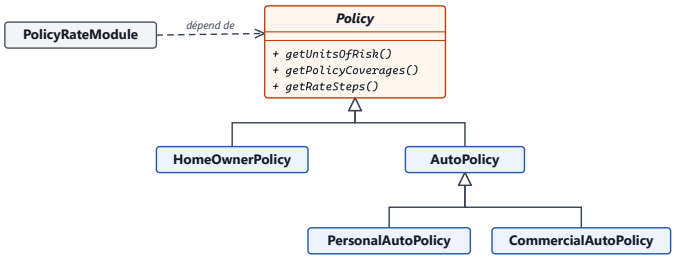


Figure 2.3 — Question de la slide 19 : PolicyRateModule dépend de Policy (abstraite) et de ses sous-classes.

SLIDE 19 : RESPECTE-T-ELLE LE LSP ? (RÈGLE DE LA SLIDE 25)

Oui, à condition que `PolicyRateModule` fonctionne **quelle que soit la sous-classe de Policy passée**, y compris les petites-filles (`CommercialAutoPolicy`) : chacune honore `getUnitsOfRisk()` , `getPolicyCoverages()` , `getRateSteps()` , et le client n'a jamais à connaître le type réel.

MÉTHODE ET PIÈGES LSP

- **Tester** : passer chaque sous-classe à un client écrit pour la mère ; en cas d'échec, **remonter** le comportement commun (`seDeplacer`) et **isoler** l'optionnel dans une interface (`Volant`), ou préférer la composition.
- LSP porte sur le **comportement**, pas la syntaxe : `Pingouin.voler()` compile. « Est-un » dans la vraie vie ne garantit pas la substituabilité.
- Une violation du LSP fait tomber l'OCP (le client ajoute des `instanceof`). Ch. 1 : dans le **Composite**, Feuille et Composite sont substituables au Composant.

I
ISP

DÉFINITION (SLIDE 21)

Il vaut mieux plusieurs interfaces spécifiques plutôt qu'une interface générale. Les clients ne doivent pas être forcés de dépendre de méthodes qu'ils n'utilisent pas.

QCM DU COURS (SLIDE 20) — QUEL EST LE PROBLÈME DE CET EXEMPLE ?

1. Trop de fonctions et de clients · **2. Des classes dépendent de méthodes dont elles n'ont pas besoin** · 3. Il n'y a pas de client A · 4. Pas de problème. **Réponse : 2** (déduite, non écrite sur la slide) : `ClientA` n'utilise que `clientAMethods()` mais toute modification pour B ou C l'impacte.

AVANT — INTERFACE MONOLITHIQUE

```
interface Service {
  void clientAMethods();
  void clientBMethods();
  void clientCMethods();
}  // ClientA voit TOUT
```

APRÈS (SLIDE 22) — UNE INTERFACE PAR CLIENT

```
interface ClientAService { void clientAMethods(); }
interface ClientBService { void clientBMethods(); }
interface ClientCService { void clientCMethods(); }
class ServiceImpl implements ClientAService,
        ClientBService, ClientCService { … }
```

Exemple 2 (slide 21) : `Service` déclare `travailler()` , `manger()` , `nager()` ; `Manager` , `Développeur` , `Comptable` l'implémentent mais **aucun n'utilise** `nager()` .
Proposition — zone masquée sur la slide On découpe en `Travailleur` , `Mangeur` , `Nageur` .

Bénéfices (slides 21, 27) : chaque classe n'implémente que ce dont elle a besoin ; plus d'interfaces **polluées**.

Pièges : ISP se raisonne **du point de vue du client** (une seule classe peut réaliser toutes les petites interfaces). ISP au niveau package = **CRP**. Une méthode qui lève `UnsupportedOperationException` signale souvent ISP *et* LSP violés.

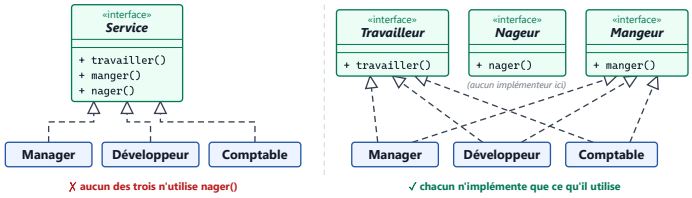


Figure 2.4 — ISP (slide 21) : Service découpée en Travailleur, Mangeur et Nageur.

D — Inversion des dépendances (DIP)

D
DIP

DÉFINITION (SLIDE 24)

Dépendre des abstractions plutôt que des implémentations. Les modules de haut niveau ne doivent pas dépendre des modules de bas niveau. **Les deux doivent dépendre d'abstractions.**

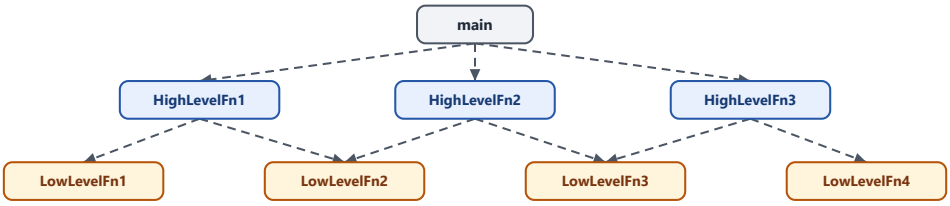


Figure 2.5 — QCM de la slide 23 : main dépend de HighLevelFn1..3, qui dépendent directement de LowLevelFn1..4.

QCM DU COURS (SLIDE 23) — QUEL EST LE PROBLÈME DE CET EXEMPLE ?

1. Trop de fonctions · 2. Les fonctions bas niveau dépendent fortement des fonctions haut niveau · **3. Les fonctions haut niveau dépendent fortement des fonctions bas niveau** · 4. Pas de problème. **Réponse : 3** (déduite, non écrite sur la slide) : toutes les flèches descendent vers le **concret** ; modifier `LowLevelFn2` remonte jusqu'à `main` (rigidité).

Pourquoi c'est grave (encadré de la slide 24) : les objets **concrets changent fréquemment** ; il faudrait modifier le haut niveau **à chaque fois**. Idée : utiliser le haut niveau indépendamment du bas niveau.

MAUVAISE CONCEPTION (SLIDE 24)

```
class EmailService {
  void envoyerEmail(String msg) { … } }
class ServiceNotification {
  private EmailService email = new EmailService();
  void envoyer(String m) { email.envoyerEmail(m); }
}  // passer au SMS ⇒ modifier ServiceNotification
```

BONNE CONCEPTION **PROPOSITION — ZONE MASQUÉE**

```
interface MessageSender { void envoyer(String m); }
class EmailService implements MessageSender { … }
class SmsService   implements MessageSender { … }
class ServiceNotification {
  private final MessageSender sender;
  ServiceNotification(MessageSender s) { sender = s; }
  void envoyer(String m) { sender.envoyer(m); }
}  // assemblage : new ServiceNotification(new
EmailService())
```

POURQUOI « INVERSION » ? LIEN AVEC L'ARCHITECTURE HEXAGONALE (SLIDE 27)

Sans DIP, la flèche va du haut niveau vers le bas niveau. Avec DIP, le bas niveau **implémente** une abstraction dont le haut niveau **dépend** : **les deux flèches pointent vers l'abstraction**, celle du bas niveau a changé de sens. « **DIP est la base de l'Architecture Hexagonale (Ports & Adaptateurs)** : le cœur métier définit les **ports**, l'infrastructure implémente les **adaptateurs**. » La logique métier reste indépendante des BDD et des frameworks UI (tests unitaires sans BDD).

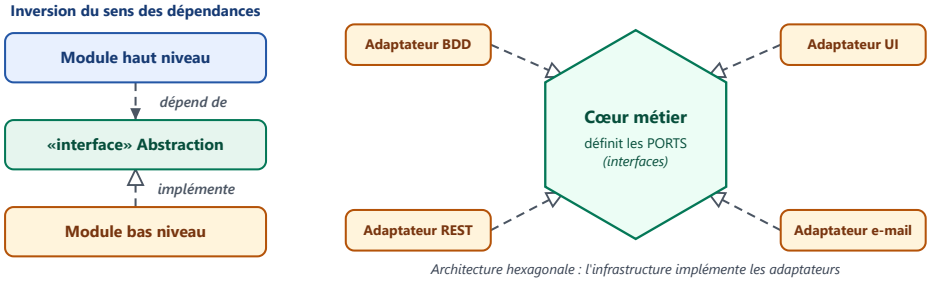


Figure 2.6 — Inversion du sens des dépendances (slide 27) et architecture hexagonale : le cœur définit les ports, l'infrastructure implémente les adaptateurs.

MÉTHODE : APPLIQUER LE DIP

1. Repérer dans le haut niveau les `new ClasseConcrète()` et attributs de type concret.
2. Définir une **interface** qui exprime le *besoin* du haut niveau (`MessageSender`).
3. La faire **implémenter** par les classes concrètes.
4. **Injecter** l'implémentation (constructeur, setter, fabrique) à l'assemblage.

PIÈGES DS

- **DIP ≠ injection de dépendances** : le principe vs une technique pour l'appliquer.
- L'interface « appartient » au haut niveau (c'est un port).
- DIP sert aussi à **casser un cycle de packages** (ADP).
- Ch. 1 : l'**Adaptateur** implémente la cible attendue ; la **Fabrique** évite le `new` du concret.

Cohésion des packages : REP, CCP, CRP

Slide 31 : « La cohésion des packages **équilibre** la réutilisabilité et la fréquence de maintenance. » Ce n'est pas une métrique statique mais un **équilibre permanent** entre facilité de partager le code et coût de sa mise à jour.

Principe	Définition (cours)	Scénario du cours → réponse
REP **Release/Reuse Equivalency**	**La granularité de la réutilisation est la granularité de la release.** Un code réutilisable se gère comme un **produit** versionné (**SemVer**), avec maintien des anciennes versions. (Slide 30 : « les éléments réutilisés ensemble doivent être livrés ensemble ».)	*Slide 35* : package « Utilitaires » non versionné ; l'équipe B change la cryptographie → l'équipe A (dates) casse ou doit retester. **REP** : A reste en **v1.x**, B passe en **v2.0**.
CCP **Common Closure**	**Les classes qui changent ensemble sont regroupées ensemble** : minimiser le nombre de packages impactés par une évolution. **= SRP au niveau package.**	*Slide 37* : loi fiscale → TVA, affichage légal, export comptable dans 3 packages = 3 redéploiements. **CCP** : un package `Facturation_Fiscale`.
CRP **Common Reuse**	**Réutiliser une classe d'un package, c'est réutiliser tout le package** : ne pas forcer les clients à dépendre de classes (et de dépendances transitives) inutiles. **= ISP au niveau package.**	*Slide 39* : la « Calculatrice Mobile » importe « Core » (SQL + maths) pour les maths → drivers SQL, compilation longue, binaire énorme, **mises à jour forcées**.

Slide 40	Focus	Taille favorisée	SOLID
REP	Réutilisation produit	petite	—
CCP	Maintenance	**grande**	SRP
CRP	Dépendances ciblées	petite	ISP

LE DILEMME DE L'ARCHITECTE (SLIDES 41-42)

REP et CRP poussent à **réduire** les packages, CCP à les **agrandir**. « Il est **mathématiquement impossible de satisfaire les 3 principes simultanément** » : on garde **2 sur 3** et on assume le coût du 3e.

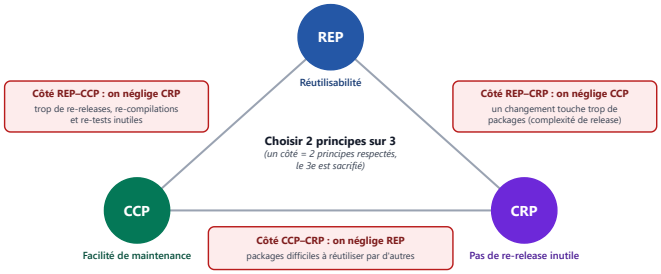


Figure 2.7 — Triangle de tension : chaque côté respecte deux principes et sacrifie le troisième.

STRATÉGIE SELON LA MATURITÉ (SLIDES 43-44)

Startup en MVP : CCP ou REP/CRP ? Et 20 micro-packages dès le 1er mois ? → **Favoriser le CCP au démarrage** : le métier change sans cesse, 20 micro-packages imposeraient de traverser des dizaines de frontières et de coordonner de nombreuses releases.

Phase 1 (démarrage) : optimiser pour le CCP, packages plus grands. **Phase 2 (maturité, réutilisation externe)** : refactoriser vers **REP et CRP**.

CHECK-LIST DE L'ARCHITECTE (SLIDE 47)

- ☐ **REP** : ai-je versionné ce package comme un produit indépendant ?
- ☐ **CCP** : ces classes changeront-elles pour la même raison métier ?
- ☐ **CRP** : mes clients importent-ils du code dont ils n'ont pas besoin ?
- ☐ **Stratégie** : taille adaptée à la maturité (CCP ➔ REP/CRP) ?

Couplage des packages : ADP, SDP, SAP

Slide 48 : « Le couplage de packages impose **des dépendances acycliques guidées par la stabilité**. »

ADP — ACYCLIC DEPENDENCIES PRINCIPLE (SLIDES 49-51)

Le graphe de dépendances entre packages doit être un DAG (graphe orienté sans cycle). On rompt les cycles via des **interfaces** ou des **packages intermédiaires**.

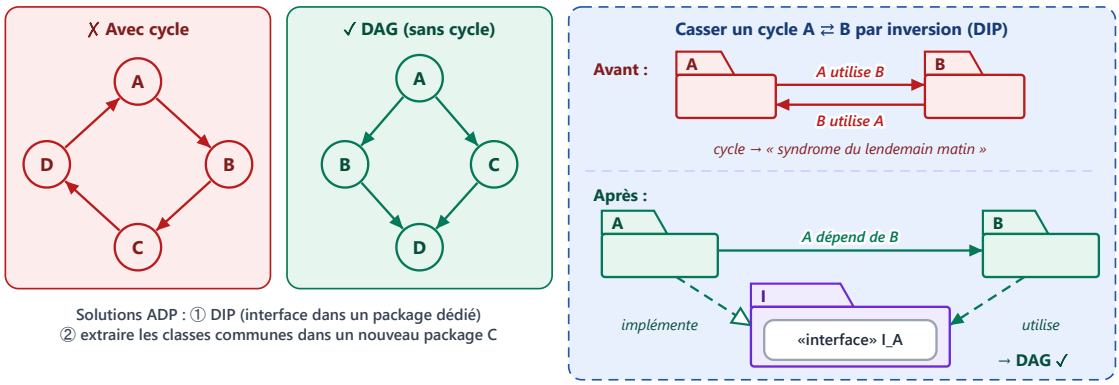


Figure 2.8 — ADP : cycle à éviter, DAG correct, et cassage d'un cycle A ⇄ B par inversion (B utilise I_A, que A implémente).

LE « SYNDROME DU LENDEMAIN MATIN »

Avec un cycle (A ➔ B ➔ A), une modification **casse les builds de tout le monde sans préavis** : on ne peut plus compiler, tester ni livrer A sans B ; ils forment de fait un seul gros package.

DEUX SOLUTIONS POUR CASSER UN CYCLE (SLIDE 51)

1. **DIP** : créer une interface (dans l'appelant ou un package d'interfaces) pour **inverser le sens de la flèche**.
2. **Nouveau package** C (*common*, *api*) avec les classes communes. La slide dit « C dépendant des deux autres » : lire « C **dont dépendent** A et B ».

SDP — STABLE DEPENDENCIES PRINCIPLE (SLIDES 49, 50, 52)

Dépendre uniquement de packages PLUS STABLES que soi (I plus faible). **Stable** = *difficile à modifier* : beaucoup de dépendants, peu de dépendances. Les **concrets** sont volatils, les interfaces changent peu.

Exemple du cours : `Copy` ne connaît que les abstractions `Reader` et `Writer` ; `KeyboardReader`, `DiskWriter` les implémentent.

PIÈGE — SLIDE 50

L'encadré « À retenir » dit « Dépendances toujours vers le haut (plus stable → moins stable) », mais le schéma montre que **le moins stable dépend du plus stable** (flèches de I = 0,7 vers I = 0,1).

Règle à écrire au DS : A → B est correct si **I(A) > I(B)**.

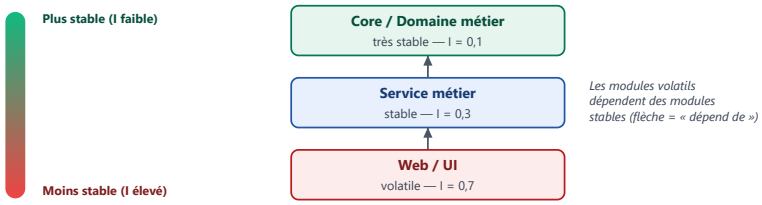


Figure 2.9 — Web/UI (I = 0,7) → Service métier (I = 0,3) → Core (I = 0,1) : chaque flèche va vers un I plus faible.

SAP — STABLE ABSTRACTIONS PRINCIPLE (SLIDES 49, 50, 53)

Un package doit être d'autant plus abstrait qu'il est stable (idéal **A + I = 1**) ; un package instable doit être concret. Stable et concret serait **rigide** ; stable et abstrait reste **extensible** (« stable ne veut pas dire figé »). Architecture idéale : noyau métier abstrait (ports) ; UI, Web, adaptateur BDD concrets qui dépendent des ports.

Couche	Caractère	A	I
Core / Domaine métier	stable + abstrait	↑	↓
Services / Logique applicative	équilibré	↔	↔
Infrastructure / Accès données	concret + volatile	↓	↑

COMMENT LES TROIS S'ARTICULENT

ADP : pas de cycle. **SDP** : les flèches vont vers le stable. **SAP** : le stable est abstrait. Donc **les dépendances vont vers l'abstrait** : c'est le DIP appliqué aux packages, et l'architecture hexagonale.

Métriques de Martin : I, A, D et séquence principale

COUPLAGES (SLIDES 49, 55, 56)

C_a (afférent) = nombre de **classes extérieures qui dépendent de** ce package (entrantes) : sa **RESPONSABILITÉ**.

C_e (efférent) = nombre de **classes extérieures dont dépend** ce package (sortantes) : sa **DÉPENDANCE**.

Mnémo : **a**fférent = ça **a**rrive vers moi ; **e**fférent = ça **e**xit de moi. N_a = classes abstraites + interfaces ; N_c = total des classes du package.

FORMULES — PAR CŒUR

$$I = C_e / (C_a + C_e) \in [0, 1]$$

0 = totalement stable (C_e = 0) ; 1 = totalement instable (C_a = 0).

$$A = N_a / N_c \in [0, 1]$$

0 = concret ; 1 = totalement abstrait.

$$A + I = 1 \quad \cdot \quad D = |A + I - 1|$$

Séquence principale = équilibre parfait ; D idéal ≈ 0, **utilisable en CI/CD** (échec du build si D dépasse un seuil).

MÉTHODE — CALCULER AU DS

1. **C_a** : compter les classes extérieures qui utilisent une classe de P (héritage, implémentation, attribut, paramètre, appel).
2. **C_e** : compter les classes extérieures utilisées par P. Chaque classe une seule fois ; l'interne ne compte pas.
3. Calculer **I**, puis **A**, puis **D**.
4. **Interpréter** : D ≈ 0 bon ; I et A petits = zone de douleur ; I et A grands = zone d'inutilité.
5. Vérifier le **SDP** (I source > I cible) et l'**ADP** (aucun cycle).

Zone de rigidité / de douleur (I = 0, A = 0) : très stable et très concret ; beaucoup de clients, rien d'abstrait, impossible à faire évoluer sans tout casser (ex. **schéma de BDD brut**).

Zone d'inutilité (I = 1, A = 1) : très instable et très abstrait ; **aucun client** (ex. **interfaces isolées**).

Piège (slide 57) : la figure dessine la zone d'inutilité comme une bande verticale ; retenez le **coin I = 1, A = 1**. Sur la figure, *LegacyDb* (I ≈ 0,15, A ≈ 0,15) a D ≈ 0,7 : zone de douleur ; CoreDomain, Services et WebUI sont sur la séquence.

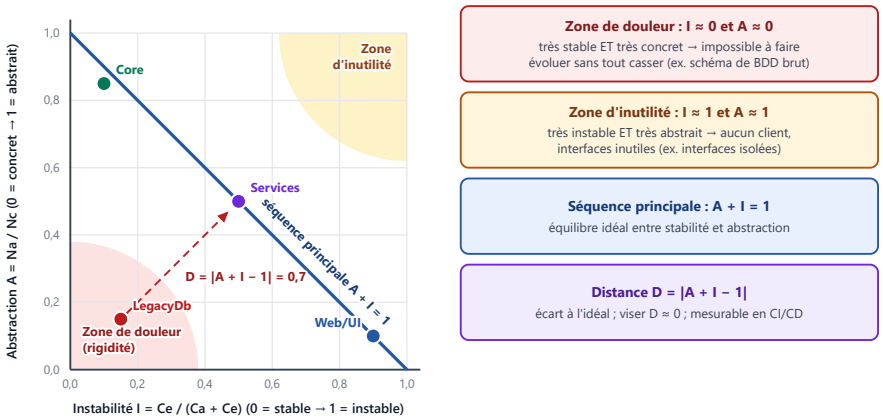


Figure 2.10 — Plan (I, A) des slides 57-58 : séquence principale, zone de douleur, zone d'inutilité et distance D.

Exemple corrigé complet

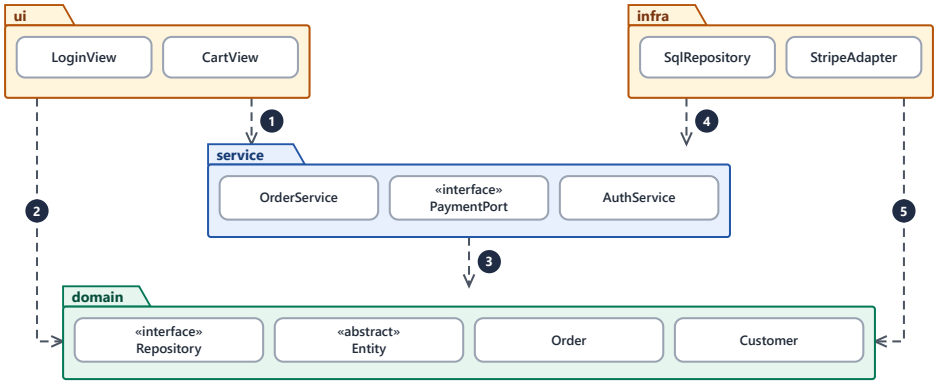


Figure 2.11 — Quatre packages (ui, service, domain, infra) ; les flèches numérotées regroupent les dépendances entre classes.

Flèche	Dépendances entre classes
① **ui → service**	`LoginView → AuthService` ; `CartView → OrderService`
② **ui → domain**	`CartView → Order`
③ **service → domain**	`OrderService → Repository, Order` ; `AuthService → Customer`
④ **infra → service**	`StripeAdapter` implémente `PaymentPort`
⑤ **infra → domain**	`SqlRepository` implémente `Repository`, hérite de `Entity`

Package	C_a (qui dépend de moi ?)	C_e (de qui je dépends ?)	I	A = N_a/N_c	D
ui	0	3 (AuthService, OrderService, Order)	3/3 = **1**	0/2 = **0**	**0**
service	3 (LoginView, CartView, StripeAdapter)	3 (Repository, Order, Customer)	3/6 = **0,5**	1/3 ≈ **0,33**	**0,17**
domain	4 (CartView, OrderService, AuthService, SqlRepository)	0	0/4 = **0**	2/4 = **0,5**	**0,5**
infra	0	3 (PaymentPort, Repository, Entity)	3/3 = **1**	0/2 = **0**	**0**

INTERPRÉTATION

- **ADP** : aucun cycle ✓. **SDP** : ui (1) → service (0,5) → domain (0) ; infra (1) → service et domain : chaque flèche va vers un I plus faible ✓.
- **ui** et **infra** (I = 1, A = 0) sont sur la séquence : détails concrets et volatils, c'est normal.
- **domain** (I = 0, A = 0,5, D = 0,5) est très stable mais à moitié concret : il tend vers la zone de douleur. Le SAP demande d'augmenter A (exposer des interfaces plutôt que `Order` et `Customer`), sinon surveiller D en CI/CD.

À retenir pour le DS

Slide 54 — Phase 1 : démarrage (vélocité, CCP)	Phase 2 : structuration du cœur (SOLID, DIP)	Phase 3 : maturité (REP / CRP)
Regrouper les classes qui évoluent ensemble ; peu de packages ; valider vite le métier.	Isoler le domaine via des abstractions stables (SAP, DIP) ; éliminer les cycles (ADP) ; tests unitaires sans BDD.	Packages ou microservices versionnés SemVer (REP) ; interfaces spécifiques (ISP, CRP) ; métriques I et A en CI/CD.

« Un bon architecte ne cherche pas la perfection immédiate, mais construit une structure capable de changer sans s'effondrer. » (slide 60)

DÉFINITIONS ET CORRESPONDANCES

- **SRP** : une seule raison de changer. **OCP** : ouvert à l'extension, fermé à la modification. **LSP** : sous-classe substituable sans altérer le client. **ISP** : interfaces spécifiques par client. **DIP** : haut et bas niveau dépendent d'abstractions.
- **CCP** ↔ **SRP**, **CRP** ↔ **ISP**, REP : aucun équivalent. REP, CRP : petits packages ; CCP : grands ; on choisit **2 sur 3** (démarrage : CCP ; maturité : REP, CRP).
- **ADP** : DIP ou nouveau package. DIP = base de l'**architecture hexagonale**.
- Haute cohésion + faible couplage ≈ **−40 %** de coûts de maintenance.

FORMULES ET ZONES

- C_a : qui dépend de moi (responsabilité) ; C_e : de qui je dépends (dépendance).
- **$I = C_e / (C_a + C_e)$** (0 stable, 1 instable) ; **$A = N_a / N_c$** (0 concret, 1 abstrait).
- **$A + I = 1$** ; **$D = |A + I - 1|$**, idéal 0.
- Douleur : I = 0, A = 0 (ex. schéma BDD). Inutilité : I = 1, A = 1 (interfaces isolées).
- SDP : A → B si **I(A) > I(B)**. SAP : stable ⇒ abstrait.

PIÈGES CLASSIQUES

- Risques 5, 6, 7 numérotés différemment selon les slides : **citez les noms**.
- I a **C_e au numérateur** (ne pas inverser C_a et C_e). « Stable » = difficile à changer (beaucoup de dépendants), pas « sans bug ».
- SDP : malgré la slide 50, on **dépend du plus stable**. Zone d'inutilité = **coin** I = 1, A = 1.
- LSP = comportement (un code qui compile peut le violer) ; DIP ≠ injection ; SOLID partout « au cas où » = complexité inutile.

Exercices corrigés

Exercice 1 — Identifier le risque. a) Ajouter un champ à `Client` oblige à modifier 12 classes. b) Un singleton global ajouté « pour aller vite » au lieu d'un paramètre. c) Une interface générique à 4 niveaux d'héritage pour un export à un seul format. d) Le calcul de TVA recopié dans 4 écrans. e) Le module PDF, qui lit directement la configuration du projet, n'est pas réutilisable ailleurs.

CORRIGÉ

a) **Rigidité**. b) **Viscosité** (abaisser le coût du bon chemin). c) **Complexité inutile** (YAGNI). d) **Répétition inutile** (DRY). e) **Immobilité** (DIP : dépendre d'une interface de configuration).

Exercice 2 — OCP / DIP. Corrigez : `double remise(Client c) { if (c.type.equals("STANDARD")) return 0; else if (c.type.equals("FIDELE")) return 0.05; else if (c.type.equals("VIP")) return 0.15; return 0; }`

CORRIGÉ

Chaque nouveau type oblige à modifier la méthode : **OCP violé**. On remplace l'aiguillage par du polymorphisme (patron **Stratégie**, ch. 1) et on injecte la politique (DIP) :

```
interface PolitiqueRemise { double taux(); }
class RemiseFidele implements PolitiqueRemise { public double taux() { return 0.05; } }   // idem Standard, Vip
class Client { private final PolitiqueRemise p;
               Client(PolitiqueRemise p) { this.p = p; }
               double remise() { return p.taux(); } }   // nouveau type = nouvelle classe
```

Exercice 3 — LSP. `Carre extends Rectangle` redéfinit `setLargeur(l)` pour modifier aussi la hauteur. Ce client est-il correct pour toute sous-classe : `void test(Rectangle r) { r.setLargeur(5); r.setHauteur(4); assert r.aire() == 20; }` ?

CORRIGÉ

Non : avec un `Carre`, `setHauteur(4)` remet la largeur à 4, l'aire vaut 16 et l'assertion échoue. `Carre` n'est pas substituable : **LSP violé**, bien qu'un carré « soit un » rectangle en mathématiques. Correction : pas d'héritage entre formes mutables ; une abstraction commune `Forme { double aire(); }` (comme `Oiseau.seDeplacer()`) ou des formes immuables.

Exercice 4 — QCM d'examen du cours (slide 59). Q1. C_a = 15 et C_e = 0 : instabilité ? Q2. Pourquoi éviter de modifier la zone de rigidité (I = 0, A = 0) ? Q3. Quel principe de package correspond au SRP appliqué aux composants ?

CORRIGÉ

Q1. I = 0 / 15 = **0** : totalement stable. **Q2.** De nombreux clients en dépendent et il est **100 % concret** : toute modification est très risquée (la slide écrit « est dépendant de nombreux clients » : lire « de nombreux clients dépendent de lui »). **Q3.** Le **CCP**.

Exercice 5 — « GlobalApp » (exercice du cours, slides 45-46). Tout réside dans `GlobalApp` (UserAuth, UserUI, User, Product, ProductDB, couplés). L'équipe Sécurité veut importer **uniquement** `UserAuth`. Avec le dilemme de l'architecte, proposez un plan immédiat et un plan cible.

CORRIGÉ

Plan immédiat (CCP) : regrouper par axe de changement : `Users` {User, UserAuth, UserUI} et `Products` {Product, ProductDB}. **Plan cible (REP + CRP)** : extraire `UserAuth` dans un package indépendant **versionné** (v1.0.0) pour ne pas imposer l'UI à l'équipe Sécurité. *Dilemme* : au démarrage on privilégie CCP ; quand un client externe apparaît, on paie un package de plus pour gagner REP et CRP.

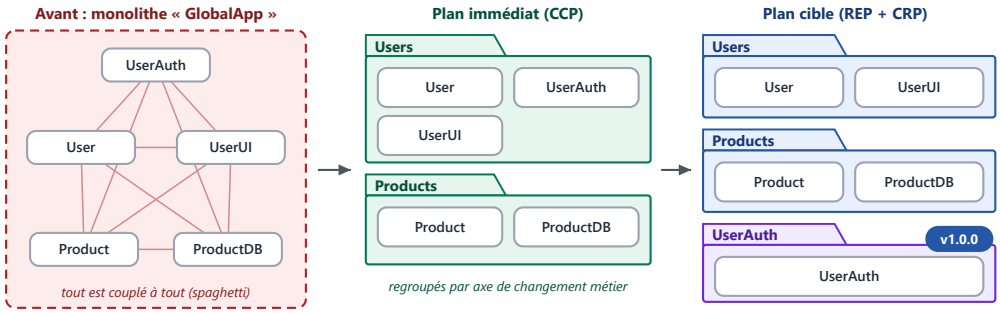


Figure 2.12 — Corrigé du cours : monolithe, découpage CCP, puis extraction REP/CRP de UserAuth en v1.0.0.

Exercice 6 — Calculs et SDP. a) C_a = 5, C_e = 15 : I ? b) 8 classes dont 6 abstraites ou interfaces, avec l'I de a) : A, D, diagnostic ? c) `web` (C_a = 0, C_e = 6), `metier` (6, 2), `outils` (1, 3), avec web → metier → outils : SDP respecté ?

CORRIGÉ

a) I = 15/20 = **0,75**. b) A = 6/8 = **0,75** ; D = |0,75 + 0,75 − 1| = **0,5** : vers la **zone d'inutilité** (trop d'abstractions pour peu de clients).
c) I(web) = **1**, I(metier) = 2/8 = **0,25**, I(outils) = 3/4 = **0,75**. web → metier ✓ ; **metier → outils** ✗ (le stable dépend de l'instable). Correction : interface dans `metier`, implémentée par `outils` (DIP, la flèche s'inverse).

Exercice 7 — Casser un cycle (ADP). `commande` utilise `facturation.Facture` ; `facturation` utilise `commande.Commande` pour lire les lignes. Deux solutions ?

CORRIGÉ

Le cycle viole l'**ADP**. 1) **DIP** : interface `Facturable { List<Ligne> lignes(); }` dans `facturation`, implémentée par `Commande` : il ne reste que commande → facturation. 2) **Nouveau package** `commun` (`Ligne`, `Montant`) dont dépendent les deux. Dans les deux cas : un DAG.

CHAPITRE 3

Architecture logicielle

Objectifs du chapitre

- définir une architecture et la **situer parmi les niveaux d'abstraction** (4 couches, 5 niveaux) ;
- distinguer **BF / BNF**, prioriser avec **MoSCoW**, formuler un **scénario de qualité mesurable** ;
- **arbitrer** entre exigences contradictoires (compromis, synchrone / asynchrone).

Ce qui tombe au DS (checklist officielle)

- Différence **BF / BNF** et métaphore de **Blanche-Neige**.
- Les **6 éléments** d'un scénario de qualité.
- Les **3 modèles de découplage** de base de données.
- Pourquoi il n'existe **pas d'architecture optimale** universelle.
- Sujet type : l'**atelier télémédecine**.

ShopExpress et définition de l'architecture

Fil rouge : le crash de ShopExpress

Le jour du **Black Friday**, **50 000 utilisateurs simultanés** arrivent sur ShopExpress : **une seule application monolithique** (Catalogue, Paiement, Recommandations imbriqués) sur **un unique serveur**, avec **une base de données centrale**. La base sature et tout s'arrête : commandes, paiement, recommandations. L'équipe ne peut pas isoler la panne, car « tout est lié ».

Questions de l'architecte (slide 5)	Réponse attendue
Bug de code ou problème d'architecture ?	**Architecture** : chaque fonction marche, c'est la *structure* (un bloc, une base, aucune isolation) qui cède.
Pourquoi un modèle de données unique partagé est-il risqué ?	**Couplage fort** (changer une table pour un module casse les autres), **goulot d'étranglement** et **point de défaillance unique**.
Comment éviter qu'une panne des recommandations bloque l'achat ?	**Découpler et isoler** : un service indépendant avec ses propres données, appelé avec un délai maximal et une réponse par défaut, ou alimenté en asynchrone par des événements (dégradation gracieuse).
Quels attributs de qualité ont été négligés ?	**Scalabilité, disponibilité, découplage** (réponse donnée sur la slide).

Ligne 4 : réponse de la slide. Lignes 1 à 3 : **Proposition de corrigé — non fournie dans le cours**.

À RETENIR

ShopExpress n'est pas un bug : c'est un échec d'architecture. Les BF étaient là, mais les BNF (scalabilité, disponibilité, découplage) avaient été ignorés. Une bonne architecture se voit surtout **pendant une crise ou une évolution**. *Lien ch. 2* : le monolithe imbriqué cumule **fragilité**, **rigidité** et **immobilité** ; la base partagée est un couplage caché, par les données.

Définition

DÉFINITION — ARCHITECTURE LOGICIELLE

L'architecture logicielle est l'**organisation globale d'un logiciel**. Elle comprend trois éléments : ① les **sous-systèmes** (la division du logiciel) ; ② les **politiques d'interaction** (les règles selon lesquelles les sous-systèmes interagissent) ; ③ les **interfaces** (les points de contact et les contrats d'échange formalisés).

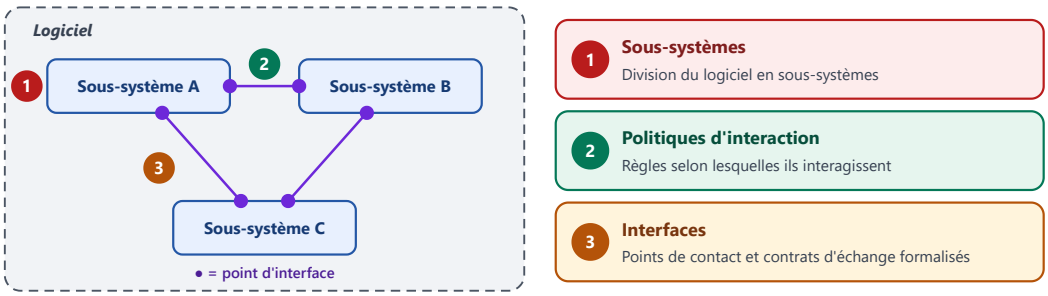


Figure 3.1 — Les trois éléments de la définition : ①sous-systèmes, ②règles d'interaction, ③interfaces (• points d'interface).

✓ CE QUE L'ARCHITECTURE ENGLOBE

- La structure des **composants majeurs**.
- Les **modèles de communication** (synchrone / asynchrone).
- L'**affectation des responsabilités** métier.
- Les **décisions difficiles à changer** plus tard.

✗ CE QU'ELLE N'EST PAS

- Le détail d'implémentation des fonctions internes.
- Le choix algorithmique de bas niveau.
- Une simple collection d'outils et de frameworks.
- Un simple schéma des dossiers du projet.

EXEMPLE — SHOPEXPRESS

Sous-systèmes : Catalogue, Commandes, Paiement, Recommandations. **Politiques d'interaction** : le Paiement est appelé en synchrone par les Commandes ; les Recommandations sont mises à jour par événements, sans bloquer l'achat. **Interfaces** : l'API `POST /paiements` et son contrat (JSON, codes d'erreur), l'événement `CommandeValidée`. Le choix entre `ArrayList` et `HashMap` dans une méthode n'en fait **pas** partie. Enjeu économique : moins de **time-to-market** et de **coûts de maintenance** à long terme.

Architecture vs design : les 4 critères de décision

« Toute architecture est du design, pas l'inverse. » L'architecture est le **sous-ensemble du design** qui regroupe les **décisions structurantes et coûteuses à changer** (base partagée, synchrone ou asynchrone, microservices). Le reste (algorithmes, structures de données, noms, patterns locaux) relève du design.

Critère	Question à se poser	Si « non » → design (local)	Si « oui » → architecture
1. Portée	Où la décision s'applique-t-elle ?	Une fonction ou une classe	Plusieurs composants ou tout le système
2. Coût de changement	Que faut-il pour revenir en arrière ?	Quelques lignes à modifier	Refaire des composants, migrer des données, renégocier des interfaces
3. Impact qualité	Un attribut de qualité en dépend-il ?	Aucun attribut touché	Performance, sécurité, scalabilité ou maintenabilité en dépendent
4. Parties prenantes	Qui doit donner son accord ?	Le développeur seul	Plusieurs intérêts à concilier : métier, exploitation, sécurité, coût

MÉTHODE — CLASSER UNE DÉCISION AU DS

1. Poser les **4 questions** dans l'ordre : portée, coût de changement, impact qualité, parties prenantes.
2. Répondre **oui / non** avec une justification courte ; pour la qualité, nommer l'attribut touché.
3. Conclure : **un seul « oui » suffit** → **architecture** ; quatre « non » → **design**.

PIÈGES CLASSIQUES

- Pas besoin de 4 « oui » : **un seul** suffit.
- Un framework n'est pas « l'architecture » ; le *style* qu'il impose (microservices, événementiel) l'est.
- *Pattern local* (une Fabrique dans un module) = design ; *style architectural* (couches, microservices, event-driven) = architecture.

Décision (exemples du cours)	Portée	Coût	Qualité	Parties prenantes	Verdict
Tri rapide ou tri par fusion dans une fonction	non	non	non	non	**Design**
Base de données partagée entre 3 services	oui	oui	oui	oui	**Architecture**
Passer au chiffrement de bout en bout	oui	oui	oui (sécurité, performance)	oui	**Architecture**

Pourquoi une architecture ? Bénéfices, cycle de vie, architecte

Situation du cours : un projet de **6 mois**, **20 développeurs**, plusieurs parties prenantes. « Et si on codait sans architecture ? » L'architecture répond par des **décisions structurantes prises tôt**, qui agissent comme **3 leviers** et apportent **3 bénéfices**.

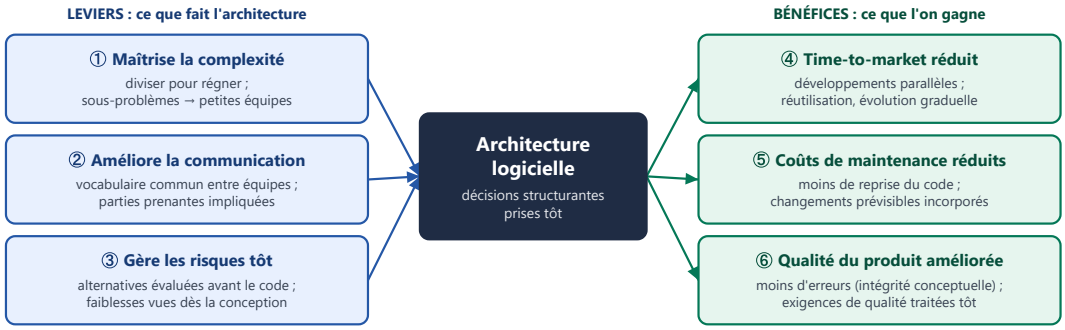


Figure 3.2 — Les 3 leviers (maîtriser la complexité, améliorer la communication, gérer les risques tôt) et les 3 bénéfices (time-to-market, coûts de maintenance, qualité).

Cycle de vie (Schéma 1 du cours)

Sans architecture (peu de parties prenantes) : exigences et qualité mènent directement à un accord, puis au développement. **Avec architecture** (beaucoup d'acteurs) : parties prenantes, exigences et qualité **convergent vers l'architecture** avant l'accord. **L'architecture formalise une vision commune, puis un développement maîtrisé.**

L'architecte : « le Sage »

Il se place entre le **métier** (besoins, budget, délais, priorités) et la **technique**, et traduit les besoins en **décisions de structure**. Qualités : **vision globale** et **impartialité**. Rôle : **arbitre entre sécurité, performance et coût**.

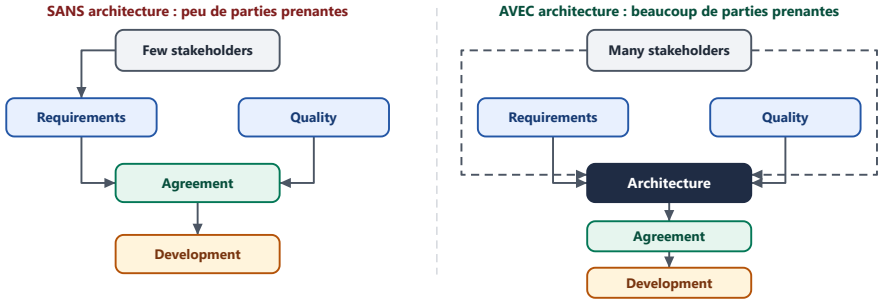


Figure 3.3 — Schéma 1 : sans architecture (peu de parties prenantes) et avec architecture (beaucoup de parties prenantes). Libellés en anglais comme sur la figure du cours.

LES 5 TÂCHES DE L'ARCHITECTE (À RÉCITER)

① Capturer les **BNF** → ② choisir **styles et patrons** → ③ définir **API et contrats** → ④ valider par **POC** (*Proof of Concept*) → ⑤ **encadrer les équipes**. On commence par les BNF, pas par le code : ce sont eux qui guident le choix du style.

Niveaux d'abstraction : 4 couches, 5 niveaux, familles de produits

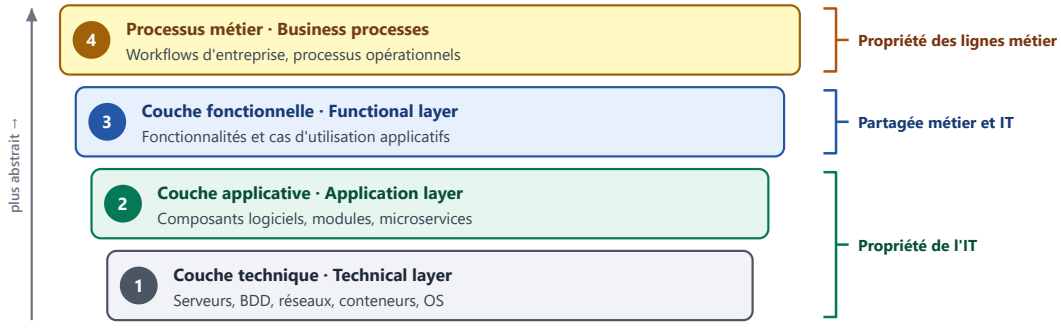


Figure 3.4 — Schéma 2 : les 4 couches, de la plus concrète (1, technique) à la plus abstraite (4, processus métier), avec leur propriétaire.

4 COUCHES — L'ORDRE ET LES PROPRIÉTAIRES

Du plus concret au plus abstrait : **T-A-F-M** = **T**echnique → **A**pplicative → **F**onctionnelle → **M**étier. Propriétaires : processus métier → **lignes métier** ; couche fonctionnelle → **partagée** métier / IT ; couches applicative et technique → **IT**. Une fonctionnalité est réalisée par un ou plusieurs composants applicatifs, eux-mêmes déployés sur l'infrastructure technique.

ShopExpress : *serveurs, BDD, Kubernetes* (1) → *services Catalogue et Paiement* (2) → *« passer une commande », « payer »* (3) → *processus « vente en ligne et livraison »* (4).

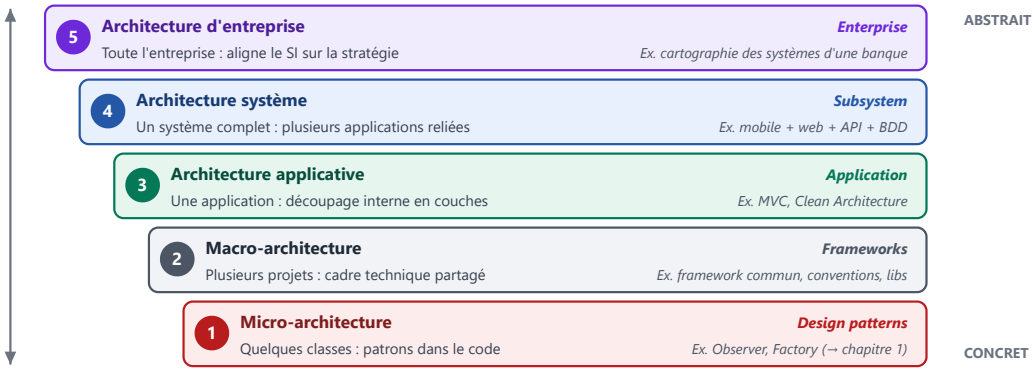


Figure 3.5 — Les 5 niveaux d'architecture, du plus abstrait (5, entreprise) au plus concret (1, micro-architecture).

5 NIVEAUX — RETENIR L'ORDRE

Du plus concret (1) au plus abstrait (5) : **Mi · Ma · App · Sys · Ent**.

Micro (patrons du ch. 1) → **Macro** (frameworks partagés par plusieurs projets) → **Applicative** (couches présentation / métier / données, MVC, Clean ou hexagonale, qui appliquent le DIP du ch. 2) → **Système** (plusieurs applications) → **Entreprise** (stratégie alignée avec le SI). Critère : la **portée**.

PIÈGES

- 4 couches = tranches d'un système (« qui possède quoi ») ; 5 niveaux = portée du zoom architectural. Ne pas les confondre.
- Dans les exercices, le rang **1 = le plus concret**.
- La **macro**-architecture est *plus concrète* que l'architecture applicative, malgré son nom.

Familles de produits : l'architecture porte la variabilité

ARCHITECTURE DE FAMILLE (EX. IPHONE / IPOD / IPAD)

Toute la famille partage **une architecture commune** : les points communs sont **figés** ; les variantes (écran, OS, messagerie, connectivité…) sont des **points de variation configurables**, décrits par un modèle de caractéristiques (● obligatoire, ○ optionnel, ex. la caméra).

LES 6 PROPRIÉTÉS À ASSURER

Configurabilité, **scalabilité**, **performance**, **adaptabilité**, **compatibilité**, **généricité**.

Mnémo : **C-S-P-A-C-G**, « *Chaque Smartphone Partage une Architecture Commune Générique* ».

Découplage : trois versions du même système

Le cours compare trois architectures qui offrent **exactement les mêmes fonctionnalités** : un client utilise trois services A1, A2 et A3.

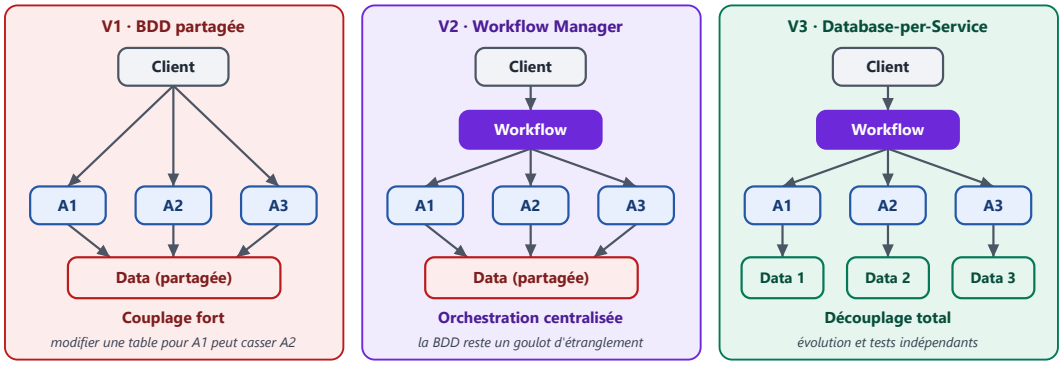


Figure 3.6 — V1 : base partagée → couplage fort. V2 : orchestrateur (workflow manager) → orchestration centralisée, mais la base reste un goulot. V3 : une base par service (Database-per-Service) → découplage total.

Exercice du cours (corrigé officiel) : notez chaque propriété de 1 à 3 points. Colonne « Justification » ajoutée.

Propriété	V1 · BDD partagée	V2 · Workflow	V3 · BDD par service	Justification
Maintenabilité	●○○	●●○	●●●	En V3, on modifie un service et sa base sans toucher aux autres
Sécurité	●○○	●●○	●●●	En V3, chaque service n'accède qu'à ses données (cloisonnement)
Découplage	●○○	●●○	●●●	V2 découple le client ; V3 découple aussi les données
Testabilité	●○○	●●○	●●●	En V3, chaque service se teste seul, avec sa base

Propriété	V1 · BDD partagée	V2 · Workflow	V3 · BDD par service	Justification
Performance	●●●	●●○	●●○	V1 : accès directs. V2 et V3 : sauts réseau supplémentaires

À RETENIR (ENCADRÉ DU COURS)

« Ces versions diffèrent-elles par leurs fonctionnalités ? » **Non : mêmes BF, BNF différents.** L'architecture agit surtout sur la *qualité*.

- **Gain** (V1 → V3) : maintenabilité, découplage, sécurité, testabilité.
- **Coût** : appels réseau supplémentaires, donc performance en retrait.
- **« Un BNF se gagne souvent au prix d'un autre. »** C'est le **compromis** (*trade-off*).

PIÈGES

- V3 n'est **pas** meilleure en tout : sa performance baisse (V1 a 3 points, V3 en a 2).
- En V2, l'orchestrateur découple le client, mais la **base partagée** reste le goulot.
- *Lien ch. 2* : V1 reproduit le couplage fort et la fragilité ; V3 applique le SRP et l'encapsulation à l'échelle d'un service (forte cohésion, faible couplage).

Exigences : BF / BNF, MoSCoW et attributs de qualité

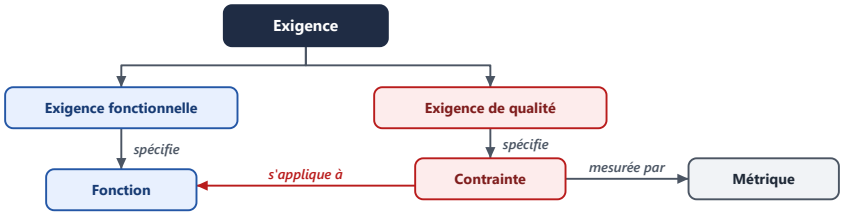


Figure 3.7 — Arbre des exigences : une exigence fonctionnelle spécifie une fonction ; une exigence de qualité spécifie une contrainte, mesurée par une métrique et appliquée à une fonction. Ex. « payer » + « en moins de 2 s », mesuré par le temps de réponse moyen.

BF — CE QUE LE LOGICIEL DOIT FAIRE

- Les actions à réaliser : la **raison d'être** du produit.
- **Visibles** par les utilisateurs finaux ; priorisées avec **MoSCoW**.

Ex. : « ajouter un produit au panier », « payer par carte ».

BNF — COMMENT IL DOIT SE COMPORTER

- Des contraintes de qualité, **souvent invisibles**.
- Ignorées au départ, elles rendent le système **imprévisible et coûteux à modifier** ; elles **déterminent le succès architectural**.

Ex. : « 50 000 utilisateurs simultanés », « paiement en ≤ 2 s ».

BLANCHE-NEIGE ET LES 7 NAINS (CHECKLIST DS)

Blanche-Neige = les **BF** (personnage central, raison d'être). Les nains (mètre ruban, horloge, tee-shirt « SECURITY », loupe…) = les **BNF** : secondaires mais **indispensables**, un nain manquant et l'histoire est compromise. ShopExpress avait sa Blanche-Neige (catalogue, paiement), pas les nains « scalabilité » et « disponibilité ».

POURQUOI LES BNF SONT ABANDONNÉS

Ils complètent l'histoire sans être le cœur de la démo ; ils paraissent moins importants ; on ne les « voit » pas tant qu'aucun incident ne survient ; ils sautent en premier quand le **planning se resserre**. La facture tombe **six mois plus tard** : dette technique, failles, impossibilité de scaler.

Question de découverte 2 : les plus abandonnés sous pression sont la **portabilité** et la **documentation**, avec l'**auditabilité** et la **testabilité**.

Prioriser avec la méthode MoSCoW

Lettre	Catégorie	Définition (cours)	Exemple ShopExpress
M	**Must have**	À satisfaire **absolument** dans la solution finale ; sans eux, elle n'a pas de sens	Passer commande, payer, tenir la charge du Black Friday
S	**Should have**	Prioritaire, à ajouter **si possible** dans la version courante	Suivi de livraison en temps réel
C	**Could have**	Souhaité, mais **non critique** en cas de manque de temps	Recommandations personnalisées
W	**Won't have**	**Écarté du livrable actuel**, avec l'accord des parties prenantes	Paiement en cryptomonnaie

MÉTHODE ET PIÈGES MOSCOW

Sans elle, la solution a-t-elle encore un sens ? Non → **M**. Importante et faisable dans cette version → **S**. Un « plus » sacrifiable si le temps manque → **C**. Reportée avec l'accord des parties prenantes → **W**. *Pièges* : W = « pas *pour cette version* », pas « jamais » ; les « o » ne veulent rien dire ; un BNF peut être **Must** (sécurité des données de santé).

Les 10 attributs de qualité

Attribut	Définition (cours)	À retenir (cours)
Groupe 1 : évolution et exploitation		
Maintenabilité	Facilité de changer le système (correction, nouvelles fonctions)	Plus il est maintenable, moins le coût de possession est grand
Portabilité	Facilité d'installation sur plusieurs plateformes	Indépendance vis-à-vis de l'OS et du matériel
Fiabilité	Capacité à maintenir sa performance dans le temps	Un logiciel non fiable tombe en panne fréquemment
Scalabilité	Gérer une large variété de tailles de système	Utilisable pour 5 comme pour 5 000 utilisateurs
Flexibilité	Se plier sans se briser face aux changements de besoins	Réutiliser, déployer et tester plus facilement
Groupe 2 : traçabilité, temps et usage		
Auditabilité	Capacité à retrouver les causes des erreurs	Journaliser un maximum, lisible et extractable, sans nuire à la performance
Documentation	Maintenir une documentation du système à jour	Tracer les changements ; suivre besoins et spécifications sur tout le cycle de vie
Performance	Respect des contraintes temporelles du logiciel	Identifier les tâches « time-sensitive », tenir les attentes des utilisateurs
Sécurité	Authentification, confidentialité, intégrité, non-répudiation	**Plusieurs propriétés à définir**, pas une seule
Utilisabilité	Efficacité des utilisateurs avec le système	Facilité d'apprentissage du système

MÉMORISER LES 10 ATTRIBUTS

Groupe 1 : **M-P-F-S-F**, « *Mon Portable Fonctionne Sans Fil* ». Groupe 2 : **A-D-P-S-U**, « *Avec Des Piles Sans Usure* ».

Typologie des BNF et contraintes du projet

Développement	Exécution	Conception
Observés en codant et en faisant évoluer	*Mesurables en temps réel, en production*	*Choix qui régissent l'organisation interne*
Maintenabilité	Performance	Modularité
Testabilité	Sécurité	Découplage
Portabilité	Disponibilité	Extensibilité
Réutilisabilité	Fiabilité	Scalabilité

Contrainte	Exemples (cours)
Techniques	Infrastructure imposée, dette technique, compatibilité legacy
Métier	Budget limité, time-to-market, retour sur investissement
Légales	RGPD, NIS2, souveraineté des données, ISO 27001
Organisationnelles	Compétences de l'équipe, gouvernance, culture DevOps

PLACER UN BNF, DISTINGUER CONTRAINTE ET BNF

Constaté en codant ou en faisant évoluer → **développement** ; mesurable en production → **exécution** ; propriété de l'organisation interne → **conception**. *Piège* : disponibilité, testabilité, réutilisabilité, modularité, découplage et extensibilité ne font *pas* partie des 10 attributs ; la scalabilité est classée en **conception**. Une **contrainte** est *imposée* et non négociable (loi, budget, infrastructure existante) ; un **BNF** est un *objectif de qualité* que l'on arbitre et mesure. Ex. : l'hébergement HDS est une contrainte légale ; « alerte en moins de 1 s » est un BNF de sécurité.

Évaluation 1 (Quiz) : le quiz du cours passe par des liens externes ; son contenu ne figure pas dans les slides.

Scénarios de qualité : rendre un BNF testable

DÉFINITION — SCÉNARIO DE QUALITÉ

Un **scénario de qualité** décrit un BNF de façon structurée, en **6 éléments**. Il transforme un souhait vague (« le site doit être rapide ») en exigence **testable et mesurable**. **Un scénario rend un BNF testable.**

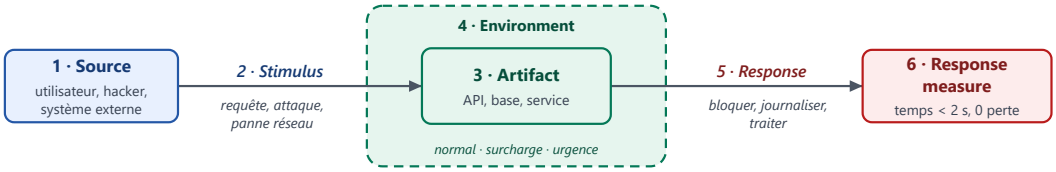


Figure 3.8 — Schéma 4 : une source envoie un stimulus à un artefact, placé dans un environnement ; l'artefact produit une réponse, évaluée par une mesure.

N°	Élément	Question à se poser	Exemples (cours)
1	**Source**	Qui ou quoi déclenche ? (origine)	Utilisateur, hacker, système externe
2	**Stimulus**	Quel événement arrive ?	Requête, attaque, panne réseau
3	**Artifact (artefact)**	Quelle partie du système est sollicitée ?	API, base, service
4	**Environment**	Dans quelles conditions ?	Normal, surcharge, urgence
5	**Response**	Que fait le système ?	Bloquer, journaliser, traiter
6	**Response measure**	Quel critère chiffré prouve la réussite ?	Temps < 2 s, 0 perte

Scénarios du cours (à connaître) :

	Source	Stimulus	Artifact	Environment	Response	Measure
Performance	Utilisateur légitime	Initie une transaction de paiement	Service de paiement et API Gateway	Fonctionnement normal	Transaction traitée avec succès	Temps de réponse moyen ≤ 2 s
Sécurité	Hacker externe	Tente de supprimer ou modifier des données confidentielles	Base de données et service de données	Serveur en surcharge	Accès bloqué, trace dans l'audit log	0 donnée perdue, alerte < 1 s
Disponibilité proposition	Panne matérielle interne	Une instance du service Commandes tombe	Service Commandes et sa base	Pic du Black Friday	Trafic basculé vers les instances saines ; panne journalisée, alerte	Disponibilité ≥ 99,9 % ; bascule < 30 s ; 0 commande perdue

La ligne « Disponibilité » répond à l'exercice du cours « rédigez un scénario équivalent pour la disponibilité de ShopExpress » : **Proposition de corrigé — non fournie dans le cours**.

MÉTHODE — RÉDIGER UN SCÉNARIO AU DS

1. Choisir **un** attribut et **un** cas précis.
2. Remplir les 6 cases dans l'ordre (Source → … → Measure), en tableau.
3. Vérifier que la **mesure est chiffrée** (temps, %, nombre).
4. Relire : « Quand *[source]* *[stimulus]* sur *[artefact]* en *[environnement]*, le système *[réponse]*, mesuré par *[mesure]*. »

PIÈGES CLASSIQUES

- Une **mesure non chiffrée** (« rapide », « sûr ») rend le scénario non testable.
- **Réponse** (l'action du système) ≠ **mesure** (le critère de réussite).
- Ne pas oublier l'**environnement** : normal ou surcharge ne donnent pas les mêmes exigences.
- « *Response measure* » (schéma) = « *Measure* » (cartes).

Compromis : pas d'architecture parfaite, synchrone vs asynchrone

« Une architecture n'est ni bonne ni mauvaise en soi : elle est plus ou moins adaptée à un objectif donné. »

L'architecte n'optimise pas, il arbitre.

Tensions du cours : **sécurité ↔ performance** (le chiffrement et les contrôles ralentissent) ; **découplage ↔ complexité** (les microservices augmentent la complexité opérationnelle et le coût). De V1 à V3, on gagne en découplage mais on perd en performance.

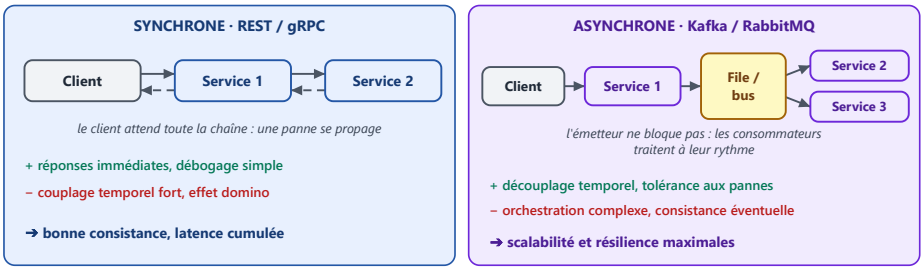


Figure 3.9 — Synchrone (REST / gRPC) : requête/réponse en chaîne. Asynchrone (Kafka / RabbitMQ) : file ou bus, diffusion vers plusieurs consommateurs.

Synchrone : le client attend toute la chaîne → *bonne consistance, latence cumulée*. **Asynchrone** : l'émetteur ne bloque pas → *scalabilité et résilience maximales*, mais consistance éventuelle.

MÉTHODE — CHOISIR AU DS

Synchrone si l'appelant a besoin de la réponse *tout de suite* pour continuer (autorisation de paiement, vérification de stock) ou si une *consistance forte* est exigée.

Asynchrone si le traitement est long, s'il y a *plusieurs consommateurs*, des *pics* ou un flux continu, si une panne *ne doit pas se propager*, et si la consistance éventuelle est acceptable.

Toujours **justifier par les attributs de qualité** touchés.

EXEMPLE — SHOPEXPRESS

- **Synchrone** : autoriser le paiement par carte (le client attend « accepté / refusé »).
- **Asynchrone** : publier `CommandeValidée` ; e-mail, stock, recommandations et statistiques consomment l'événement à leur rythme.

Si les recommandations tombent, l'achat n'est pas bloqué : c'est ce qui manquait au Black Friday.

PIÈGES

- **Consistance éventuelle** : les données *finissent* par être cohérentes ; ce n'est pas une incohérence définitive.
- En synchrone, les latences **s'additionnent** et une panne en bout de chaîne remonte jusqu'au client.
- L'asynchrone n'est pas « gratuit » : orchestration et débogage deviennent complexes.

LIEN AVEC LE CHAPITRE 1

Le bus d'événements est la version **distribuée et asynchrone de l'Observateur** : le sujet devient un producteur, les observateurs des consommateurs abonnés (couplage faible). Un message en file rappelle le patron **Commande** : une requête encapsulée, exécutée plus tard.

Discussion en groupe : les 4 questions du cours

Question	Réponse attendue Proposition — non fournie dans le cours
Q1. Pas d'architecture optimale ? Motivation et valeur de ce point de vue.	Une architecture n'est adaptée qu'à **un objectif** (BNF priorisés, contraintes, contexte) et les BNF sont **en tension**. Valeur : oblige à **expliciter les priorités** (MoSCoW, scénarios mesurables), évite la sur-ingénierie (des microservices pour 3 développeurs), fait de l'architecte un **arbitre** qui documente ses compromis.
Q2. Synchrone vs asynchrone : quels compromis ?	**Synchrone** : gagne en consistance, simplicité de débogage, réponse immédiate ; perd en disponibilité (effet domino), scalabilité, latence (cumulée). **Asynchrone** : gagne en scalabilité, résilience, découplage temporel ; perd en consistance (éventuelle), simplicité, testabilité.
Q3. Deux exigences contradictoires ?	**Sécurité ↔ performance** : chiffrer et contrôler chaque accès ralentit, alors qu'on veut un paiement en ≤ 2 s. Aussi : découplage ↔ performance, auditabilité ↔ performance, découplage ↔ coût.
Q4. Aspects architecturaux qui affectent la performance ?	Nombre de **sauts réseau** (services, orchestrateur, gateway) ; **mode de communication** ; **gestion des données** (base goulot, agrégation) ; **sécurité** (chiffrement) ; **journalisation** ; **déploiement** (un serveur ou réplication).

Tendances 2026 : monolithe modulaire, événementiel, AI-ready

Monolithe modulaire

Un seul déploiement, modules strictement découplés en mémoire : zéro latence réseau, coûts réduits, transition facile vers les microservices. Un « bon » monolithe modulaire applique les principes de paquetage du ch. 2 : cohésion (CCP / CRP), pas de cycle (ADP), dépendances vers le stable et l'abstrait (SDP / SAP).

Event-driven & serverless

Architecture réactive dont les **flux d'événements** sont le cœur d'intégration (producteurs → bus → consommateurs) ; scalabilité dynamique.

Quiz 3, Q3 (cours) : avantage principal du monolithe modulaire sur les microservices ? **Réponse B** : il garde un **découplage logique strict** sans la complexité réseau et financière des microservices.

Architectures AI-ready	Description (cours)
Bases vectorielles & RAG	Recherche sémantique branchée sur les données de l'entreprise
Orchestration d'agents	Flottes d'agents IA autonomes et interconnectés
Routage multi-modèles	Requêtes dirigées vers un modèle local rapide ou un LLM décisionnel
Agentic fitness functions	Audit automatisé et continu des contraintes d'architecture

N°	Les 5 principes de l'architecte en 2026
1	**Qualité dès le jour 1** : sécurité et scalabilité ne s'ajoutent pas à la fin « comme un papier cadeau »
2	**Découpler pour évoluer** : séparer responsabilités et données
3	**Assumer les compromis** : toute décision arbitre des exigences contradictoires
4	**Rester pragmatique** : monolithe modulaire d'abord si l'équipe est réduite
5	**Anticiper IA & Green IT** : systèmes évolutifs, éco-conçus, AI-ready

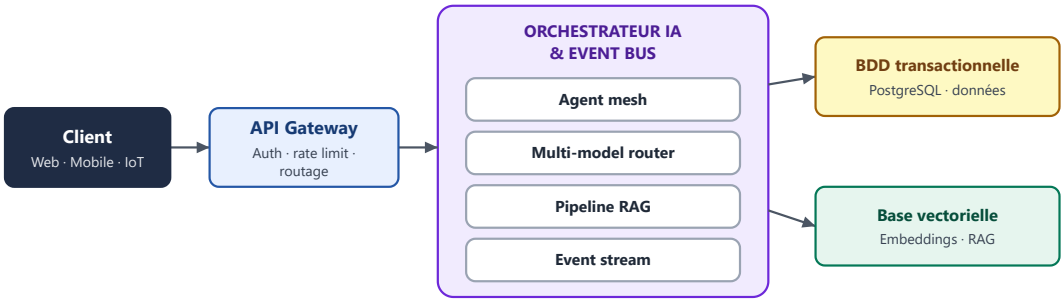


Figure 3.10 — Schéma 5 : plateforme AI-ready. Le client appelle en synchrone l'API Gateway (authentification, rate limit, routage) ; derrière, l'orchestrateur IA & event bus (agent mesh, multi-model router, pipeline RAG, event stream) s'appuie sur une BDD transactionnelle et une base vectorielle.

Autres tendances : *platform engineering* (plateformes internes, self-service, charge cognitive réduite) ; *observabilité native* (OpenTelemetry, traçabilité distribuée, détection d'anomalies par IA) ; *Green IT & FinOps* (empreinte carbone, coûts cloud, sobriété). *Lien ch. 1* : le routeur multi-modèles suit l'esprit du patron **Stratégie** (choix dynamique d'un modèle).

À retenir pour le DS

CHECKLIST OFFICIELLE DE RÉVISION (SLIDE 55), AVEC LA RÉPONSE ATTENDUE

- ☐ **Différence BF / BNF** : BF = *ce que* le logiciel doit faire (visible, raison d'être, MoSCoW) ; BNF = *comment* il doit se comporter (contraintes de qualité souvent invisibles, qui déterminent le succès architectural).
- ☐ **Blanche-Neige, et pourquoi un BNF ignoré détruit un projet** : Blanche-Neige = BF, les nains = BNF, indispensables. Un BNF ignoré donne une application imprévisible ; les « parties manquantes » sont ajoutées tard, à un coût élevé ; la facture tombe environ six mois plus tard.
- ☐ **Les 6 éléments d'un scénario** : Source, Stimulus, Artifact, Environment, Response, Response measure.
- ☐ **Les 3 modèles de découplage de BDD** : BDD partagée (couplage fort) ; workflow manager (orchestration centralisée, base encore goulot) ; BDD par service (découplage total : + maintenabilité, sécurité, testabilité ; − performance).
- ☐ **Pas d'architecture optimale universelle** : une architecture n'est adaptée qu'à un objectif ; les BNF sont contradictoires, donc l'architecte **arbitre**.

Les nombres du chapitre	Contenu
3 éléments	Définition : sous-systèmes, politiques d'interaction, interfaces
4 critères	Design vs architecture : portée, coût de changement, impact qualité, parties prenantes. Un seul « oui » suffit
3 + 3	Leviers : complexité, communication, risques. Bénéfices : time-to-market, coûts de maintenance, qualité
5 tâches	Capturer les BNF → styles et patrons → API et contrats → POC → encadrer les équipes
4 couches	Technique (1) → applicative (2) → fonctionnelle (3) → processus métier (4)
5 niveaux	Micro (1) → macro (2) → applicative (3) → système (4) → entreprise (5)
6 propriétés	Ligne de produits : configurabilité, scalabilité, performance, adaptabilité, compatibilité, généricité
4 lettres	MoSCoW : Must, Should, Could, Won't (pour cette version)
10 attributs	Maintenabilité, portabilité, fiabilité, scalabilité, flexibilité, auditabilité, documentation, performance, sécurité, utilisabilité
3 types / 4 contraintes	BNF de développement, d'exécution, de conception · contraintes techniques, métier, légales, organisationnelles
4 tendances IA	Bases vectorielles & RAG, orchestration d'agents, routage multi-modèles, agentic fitness functions
5 principes	Qualité dès le jour 1, découpler, assumer les compromis, rester pragmatique, anticiper IA & Green IT

Schémas du cours à savoir redessiner : cycle de vie (Fig. 3.3) · 4 couches (Fig. 3.4) · V1/V2/V3 (Fig. 3.6) · arbre des exigences (Fig. 3.7) · scénario de qualité (Fig. 3.8) · synchrone/asynchrone (Fig. 3.9) · plateforme AI-ready (Fig. 3.10).

PIÈGES RÉCURRENTS

Un seul « oui » suffit pour qu'une décision soit architecturale · rang 1 = le plus *concret* · les 3 versions ont les **mêmes BF** et V3 perd en performance · W = pas *dans cette version*, avec accord · une mesure de scénario est chiffrée · consistance éventuelle ≠ données fausses.

Atelier télémédecine et exercices corrigés

ATELIER D'EXAMEN DU COURS : PLATEFORME DE TÉLÉMÉDECINE 2026

Concevoir l'architecture d'une plateforme de télémédecine avec dossiers patients et assistant IA. Contraintes : **RGPD**, **hébergement de données de santé (HDS)**, **données en temps réel**. (1) Identifiez 2 BF et 3 BNF prioritaires (MoSCoW). (2) Rédigez un scénario de qualité pour la sécurité des données de santé. (3) Synchrone ou asynchrone pour les constantes vitales en temps réel ? Justifiez. (4) Schématisez les 4 couches d'abstraction.

Proposition de corrigé — non fournie dans le cours Au DS : vocabulaire du cours, compromis nommé (gain / perte), mesures chiffrées.

Q1. 2 BF et 3 BNF prioritaires, avec MoSCoW.

CORRIGÉ

Type	Exigence	MoSCoW	Justification
BF	**Téléconsultation vidéo** patient – médecin	**M**	Raison d'être (« Blanche-Neige »)
BF	**Consulter et mettre à jour le dossier patient**	**M**	Sans dossier, pas de soin
BNF	**Sécurité** : confidentialité, intégrité, authentification forte, traçabilité	**M**	RGPD + HDS : non négociable
BNF	**Performance temps réel** : alerte en moins de 5 s	**M**	Un retard met le patient en danger
BNF	**Fiabilité / disponibilité** : 24 h/24, aucune mesure perdue	**M**	Suivi continu de patients chroniques
Autres : **S** = assistant IA, scalabilité ; **C** = montres connectées ; **W** = facturation mutuelles (version suivante).			

Q2. Scénario de qualité pour la sécurité des données de santé.

CORRIGÉ

Source	Stimulus	Artifact	Environment	Response	Measure
Attaquant externe avec un jeton volé (ou interne non autorisé)	Tente de lire et d'exporter en masse des dossiers qui ne le concernent pas	API « Dossier patient » et base des dossiers (hébergement HDS)	Fonctionnement normal, heure de pointe	Accès refusé (contrôle par rôle et par patient), jeton révoqué, trace dans l'audit log, alerte à l'équipe sécurité	0 donnée divulguée ; 100 % des accès tracés ; alerte < 1 s ; chiffrement au repos et en transit

Q3. Synchrone ou asynchrone pour les constantes vitales en temps réel ?

CORRIGÉ

Asynchrone (bus d'événements type Kafka) : ① **flux continu et volumineux**, le capteur n'attend pas de réponse ; ② **plusieurs consommateurs** de la même mesure (alertes, dossier, IA, tableau de bord) ; ③ **tolérance aux pannes** : si l'IA tombe, les mesures restent dans la file, sans effet domino. *Compromis* : consistance éventuelle (léger décalage du dossier, acceptable). Le **synchrone** reste pour consulter un dossier et la visio.

Q4. Schématisez les 4 couches d'abstraction de la plateforme.

CORRIGÉ

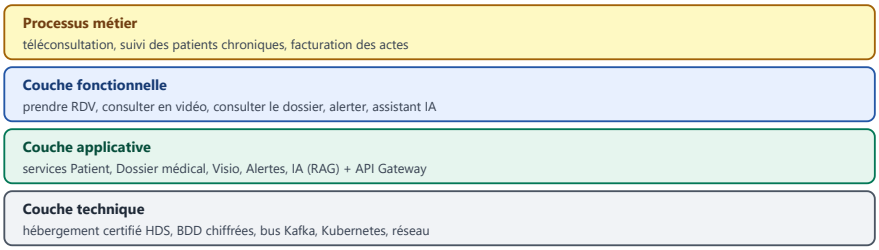


Figure 3.11 — Proposition : les 4 couches de la plateforme de télémédecine, de la plus abstraite (haut) à la plus concrète (bas).

RGPD et HDS à chaque niveau : consentement (métier), « exporter mes données » (fonctionnelle), service d'audit (applicative), hébergeur HDS et chiffrement (technique).

Exercices

Exercice 1 corrigé du cours — (1) Classez de 1 (la plus concrète) à 4 : couche applicative, fonctionnelle, métier, technique. (2) Classez de 1 (la plus concrète) à 5 : architecture d'entreprise, macro-architecture, micro-architecture, architecture système, architecture applicative.

CORRIGÉ

(1) 1 = **technique** · 2 = **applicative** · 3 = **fonctionnelle** · 4 = **métier** (**T-A-F-M**). **(2)** 1 = **micro** · 2 = **macro** · 3 = **applicative** · 4 = **système** · 5 = **entreprise** (**Mi · Ma · App · Sys · Ent**).

Exercice 2 — Design ou architecture ? (a) tri rapide ou tri fusion dans une fonction ; (b) BDD partagée entre 3 services ; (c) chiffrement de bout en bout ; (d) renommer une variable et extraire une méthode privée ; (e) passer Commandes → Stock de REST à Kafka ; (f) introduire une API Gateway devant tous les services.

CORRIGÉ

(a) **design** ; (b), (c) **architecture** corrigé du cours · (d) à (f) : Proposition — non fournie dans le cours

Décision	Portée	Coût	Qualité	Parties prenantes	Verdict
(d) Renommer, extraire une méthode	non	non	non	non	**Design** (refactoring local)
(e) REST → Kafka entre deux services	oui	oui	oui (disponibilité, consistance)	oui (exploitation)	**Architecture**
(f) API Gateway	oui	oui	oui (sécurité, perf.)	oui	**Architecture**

Exercice 3 Proposition de corrigé — non fournie dans le cours — Pour ShopExpress, classez en BF ou BNF puis priorisez (MoSCoW) : (1) payer par carte ; (2) supporter 50 000 utilisateurs simultanés ; (3) recommandations personnalisées ; (4) paiement en ≤ 2 s ; (5) application sur Android et iOS ; (6) paiement en cryptomonnaie.

CORRIGÉ

Exigence	BF / BNF	MoSCoW	Pourquoi
(1) Payer par carte	BF	**M**	Sans paiement, pas de vente
(2) 50 000 utilisateurs simultanés	BNF (scalabilité)	**M**	C'est la leçon du Black Friday
(3) Recommandations personnalisées	BF	**C**	Agréable mais non critique
(4) Paiement en ≤ 2 s	BNF (performance)	**S** ou **M**	Influence directement l'abandon de panier
(5) Android et iOS	BNF (portabilité)	**S**	Élargit l'audience
(6) Cryptomonnaie	BF	**W**	Reporté, avec l'accord des parties prenantes

Exercice 4 Proposition de corrigé — non fournie dans le cours — Identifiez les 6 éléments : « Lorsqu'un service partenaire de livraison envoie 1 000 mises à jour de statut par seconde à l'API Suivi pendant les soldes, le système les met en file et les traite toutes ; 99 % sont visibles par le client en moins de 10 s. »

CORRIGÉ

Source : service partenaire (système externe) · **Stimulus** : 1 000 mises à jour / s · **Artifact** : API Suivi · **Environment** : soldes (surcharge) · **Response** : mise en file, toutes traitées · **Measure** : 99 % visibles en < 10 s. Attributs : scalabilité, performance.

ANNEXES

Fiches de révision et sujets type DS

À relire **la veille** du DS : la fiche express rassemble tout ce qui s'apprend par cœur ; les sujets transversaux mélangent les chapitres, comme une vraie copie (à traiter au brouillon avant de lire le corrigé).

Fiche express : à savoir par cœur

CH. 1 Patrons de conception

Objectif \ Portée	Classe (héritage)	Objet (composition)
Création	Factory Method	Abstract Factory · Builder · Prototype · Singleton
Structure	Adapter (de classe)	Adapter (d'objet) · Bridge · Composite · Decorator · Façade · Flyweight · Proxy
Comportement	Interpreter · Template Method	Chain of Responsibility · Command · Iterator · Mediator · Memento · Observer · State · Strategy · Visitor

Les 23 patrons du GoF : 5 de création, 7 de structure (l'Adapter compte une seule fois), 11 de comportement.

Définition et 4 éléments

Un patron de conception est une **solution générale et réutilisable** à un **problème qui revient souvent** lors de la conception d'un logiciel. Ce n'est pas du code fini : c'est un modèle, « une recette ».

Nom → Problème → Solution → Conséquences (« Quel problème ? Quelle solution ? Quels effets ? »).

Le **GoF** (Gang of Four) : Gamma, Helm, Johnson, Vlissides, 1994, **23 patrons**.

Deux critères de classification

- **Objectif** : Création (instanciation), Structure (composition de classes et d'objets), Comportement (interactions et responsabilités).
- **Portée** : *Classe* = héritage, statique ; *Objet* = composition, dynamique (modifiable à l'exécution).

Rôles à savoir nommer

- Adaptateur : *Target* `MediaPlayer` · *Adaptee* `AdvancedMediaPlayer` · *Adapter* `MediaAdapter` · *Client* `AudioPlayer`.
- Composite : *Component* `Shape` · *Leaf* `Line`, `Rectangle` · *Composite* `ComplexShape`.
- Commande : *Command* `Order` · *ConcreteCommand* `BuyStockOrder` / `SellStockOrder` · *Receiver* `StockTrade` · *Invoker* `Agent`.
- Observateur : *Subject* · *Observer* (abstraite) · `BinaryObserver`, `OctalObserver`, `HexaObserver`.

Intention des patrons du cours

- **Fabrique** : créer des objets sans exposer la logique d'instanciation ; les sous-classes décident quelle classe instancier.
- **Singleton** : une **seule instance** et un point d'accès **global**.
- **Adaptateur** : convertir l'interface d'une classe en celle qu'attend le client.
- **Composite** : arborescence ; traiter de façon uniforme objets simples et composés.
- **Commande** : encapsuler une demande dans un objet (paramétrer, mettre en file, annuler).
- **Observateur** : dépendance un-à-plusieurs ; un changement d'état notifie et met à jour automatiquement tous les dépendants.
- **Stratégie** : famille d'algorithmes interchangeables, choisis à l'exécution.
- **Itérateur** : parcourir une collection sans exposer sa structure.
- **Fabrique + Singleton** : la fabrique dit *comment* créer ; le Singleton garantit *une seule* fabrique.

Singleton thread-safe (à savoir écrire)

```
public class Singleton {
  private static Singleton instance;
  private Singleton() { }      // privé !
  public static synchronized
         Singleton getInstance() {
    if (instance == null)        // paresseux
      instance = new Singleton();
    return instance;
  }
}
```

`static` : on appelle la méthode sans objet. `synchronized` : un seul thread à la fois, donc pas deux instances.

CH. 2 Principes de conception orientée objet

Les 7 risques (R. C. Martin) et leur remède

- **Rigidité** : un changement en entraîne dix (effet domino). Remède : découpler (DIP, OCP).
- **Fragilité** : ça casse ailleurs, sans lien apparent (couplage caché). Remède : SRP, tests.
- **Immobilité** : impossible de réutiliser un module ailleurs. Remède : DIP, ISP.
- **Viscosité** : le hack est plus simple que le bon design. Remède : *abaisser le coût du bon chemin*.
- **Répétition inutile** : copier/coller. Remède : **DRY**, abstraction commune.
- **Opacité** : code illisible (ratio effort de lecture / effort d'écriture). Remède : noms clairs, petites fonctions.
- **Complexité inutile** : sur-ingénierie, « besoin fantôme ». Remède : **YAGNI**.

Principe de réalité : *« Perfect is the enemy of good »*.

Packages : cohésion (quoi regrouper ?)

- **REP** : la granularité de la réutilisation est celle de la release (versionner en SemVer). Pas d'équivalent SOLID. Pousse vers de *petits* packages.
- **CCP** : ce qui change ensemble est regroupé ensemble. **= SRP** au niveau package. Pousse vers de *grands* packages.
- **CRP** : réutiliser une classe, c'est réutiliser tout le package. **= ISP** au niveau package. Pousse vers de *petits* packages.

Dilemme : on ne satisfait que **2 principes sur 3**. Au démarrage, CCP ; à maturité, REP et CRP.

SOLID en une ligne

- **S**RP : une classe n'a qu'**une seule raison de changer**.
- **O**CP : **ouvert à l'extension, fermé à la modification** (polymorphisme).
- **L**SP : une sous-classe doit pouvoir **remplacer** sa classe mère sans changer le comportement du client.
- **I**SP : plusieurs **interfaces spécifiques** plutôt qu'une interface « fourre-tout ».
- **D**IP : dépendre des **abstractions**, pas des implémentations ; le haut niveau ne dépend pas du bas niveau.

Les 7 risques sont les *effets observables* d'un code qui ne respecte pas SOLID.

Packages : couplage (comment dépendre ?)

- **ADP** : pas de cycle, le graphe est un **DAG**. Pour casser un cycle : DIP (interface) ou nouveau package commun.
- **SDP** : dépendre de packages **plus stables** (I plus faible).
- **SAP** : plus un package est stable, plus il doit être **abstrait** (A + I = 1).

Haute cohésion + faible couplage ≈ **−40 %** de coûts de maintenance.

FORMULES DES MÉTRIQUES DE MARTIN

Ca (afférent) = nombre de classes *extérieures qui dépendent* du package. Mesure la **responsabilité**.

Ce (efférent) = nombre de classes *extérieures dont dépend* le package. Mesure la **dépendance**.

$$I = Ce / (Ca + Ce) \in [0, 1]$$

I = 0 : totalement stable (Ca élevé, Ce = 0). I = 1 : totalement instable (Ca = 0).

$$A = Na / Nc \in [0, 1]$$

Na : classes abstraites et interfaces ; Nc : nombre total de classes du package.

$$\text{Séquence principale} : A + I = 1$$

$$D = |A + I - 1| \ (\text{idéal} : D \approx 0)$$

Zone de douleur (I = 0, A = 0) : stable et concret, donc impossible à faire évoluer (ex. schéma de BDD brut). **Zone d'inutilité** (I = 1, A = 1) : abstrait sans aucun client.

CH. 3 Architecture logicielle

Définition (3 éléments)

L'architecture est l'**organisation globale** d'un logiciel : ① les **sous-systèmes**, ② les **politiques d'interaction**, ③ les **interfaces**. Une décision est architecturale quand elle *structure le système* et qu'elle *coûte très cher à changer*.

Toute architecture est du design, mais pas l'inverse.

4 couches et 5 niveaux

Couches, du concret à l'abstrait : technique (IT) → applicative (IT) → fonctionnelle (partagée métier et IT) → processus métier (métier).

Niveaux : micro-architecture (patrons) → macro-architecture (frameworks) → applicative (couches, MVC) → système (plusieurs applications) → entreprise (SI aligné sur la stratégie).

Design ou architecture : les 4 critères

1. **Portée** : plusieurs composants ou tout le système ?
2. **Coût de changement** : migrer des données, renégocier des interfaces ?
3. **Impact qualité** : performance, sécurité, scalabilité… ?
4. **Parties prenantes** : plusieurs acteurs doivent-ils valider ?

Un seul « oui » suffit pour que la décision soit architecturale.

Rôle de l'architecte (« le Sage »)

Il traduit les besoins métier en décisions de structure et arbitre **sécurité · performance · coût**.

5 tâches : capturer les BNF · choisir styles et patrons · définir API et contrats · valider par POC · encadrer les équipes.

3 leviers (complexité, communication, risques) donnent 3 bénéfices (time-to-market, coûts de maintenance, qualité).

Exigences

- **BF** : ce que le logiciel *fait* (Blanche-Neige). **BNF** : *comment* il se comporte (les 7 nains), souvent invisibles, mais ils **déterminent le succès architectural**.
- **MoSCoW** : **M**ust (indispensable) · **S**hould (prioritaire) · **C**ould (souhaité) · **W**on't (écarté cette fois, avec accord).
- **10 attributs** : maintenabilité, portabilité, fiabilité, scalabilité, flexibilité, auditabilité, documentation, performance, sécurité, utilisabilité.
- **Types de BNF** : développement · exécution · conception. **Contraintes** : techniques · métier · légales · organisationnelles.

Découplage et compromis

- **V1**, BDD partagée : couplage fort. **V2**, workflow manager : orchestration centralisée, la BDD reste un goulot. **V3**, une BDD par service : découplage total.
- On gagne en maintenabilité, sécurité et testabilité, on perd en performance (appels réseau) : *« un BNF se gagne souvent au prix d'un autre »*.
- **Synchrone** (REST/gRPC) : réponse immédiate et cohérente, mais couplage temporel et latence cumulée. **Asynchrone** (Kafka/RabbitMQ) : découplage et résilience, mais cohérence éventuelle.

Scénario de qualité (6 éléments)

Source → Stimulus → Artefact → Environnement → Réponse → Mesure de la réponse. C'est ce qui rend un BNF **testable** : la mesure doit être chiffrée (ex. « < 2 s », « 0 perte »).

5 principes de l'architecte 2026

1. Qualité dès le jour 1.
2. Découpler pour évoluer.
3. Assumer les compromis.
4. Rester pragmatique : monolithe modulaire d'abord.
5. Anticiper IA et Green IT.

Il n'existe pas d'architecture optimale universelle : *l'architecte n'optimise pas, il arbitre.*

Correspondances entre les chapitres

Au DS, une bonne réponse **relie** les notions : un patron se justifie par un principe, un principe de package prolonge un principe de classe.

Notion	Lien	Explication en une phrase
OCP	Stratégie, Fabrique, Observateur, Commande	On ajoute une nouvelle stratégie, un nouveau produit, un nouvel observateur ou une nouvelle commande sans modifier le code existant.
DIP	Adaptateur, Fabrique, architecture hexagonale	Le client dépend d'une interface (port) ; l'adaptateur et la fabrique fournissent l'implémentation concrète.
LSP	Composite, Observateur	Feuilles et composites sont interchangeables derrière `Shape` ; tout observateur concret remplace `Observer`.
ISP	CRP	Même idée à deux échelles : ne pas forcer un client à dépendre de ce qu'il n'utilise pas.
SRP	CCP, Commande	CCP est le SRP des packages. La Commande sépare celui qui demande (invocateur) de celui qui exécute (récepteur).
ADP	DIP	On casse un cycle entre packages en insérant une interface, ce qui inverse une des dépendances.
SDP + SAP	Architecture hexagonale, 4 couches	Le noyau métier est stable et abstrait (ports) ; UI et infrastructure sont instables et concrètes et dépendent du noyau.
Observateur	Architecture événementielle, asynchrone	Publication / abonnement : l'émetteur ignore ses récepteurs. À l'échelle du système, c'est le bus d'événements (Kafka).
Cohésion / couplage	V1 → V3 (découplage BDD)	Une BDD partagée est un couplage fort entre modules ; une base par service applique la faible dépendance à l'échelle du système.

QCM éclair (corrigé)

SUJET D'ENTRAÎNEMENT · QUESTIONS 1 À 3 TIRÉES DU COURS (P. 59 DU CH. 2), LES AUTRES CONÇUES POUR LA RÉVISION

#	Question	Réponse	Justification
1	Un package a Ca = 15 et Ce = 0. Quelle est son instabilité ?	**I = 0**	I = 0/(15+0). Package totalement stable, très responsable.
2	Pourquoi éviter de modifier un package de la zone de rigidité (I = 0, A = 0) ?	Il est très utilisé et 100 % concret	Beaucoup de clients en dépendent et rien n'est abstrait : toute modification les casse.
3	Quel principe de package correspond au SRP ?	**CCP**	Regrouper les classes qui changent pour les mêmes raisons.
4	Le Singleton est un patron de quel type et de quelle portée ?	Création · objet	Il contrôle l'instanciation, et l'instance unique est un objet.
5	Quel patron convertit une interface incompatible en celle attendue par le client ?	**Adaptateur**	Structurel ; `MediaAdapter` adapte `AdvancedMediaPlayer` à `MediaPlayer`.
6	Un `switch` sur un type, à modifier pour chaque nouveau cas, viole…	**OCP**	Le code n'est pas fermé à la modification. Remède : polymorphisme (Stratégie).
7	`Pingouin extends Oiseau` lève une exception dans `voler()`. Principe violé ?	**LSP**	La sous-classe ne peut pas remplacer la mère. Remède : `seDeplacer()` + interface `Volant`.
8	A = 0,8 et I = 0,1. Que vaut D ? Interprétation ?	**D = 0,1**	|0,8 + 0,1 − 1| = 0,1 : proche de la séquence principale. Bon noyau, stable et abstrait.
9	Choisir entre tri rapide et tri fusion dans une fonction : design ou architecture ?	**Design**	Les 4 critères donnent « non » : portée locale, coût faible, pas d'attribut de qualité global, le développeur décide seul.
10	Citer les 6 éléments d'un scénario de qualité.	S · St · A · E · R · M	Source, stimulus, artefact, environnement, réponse, mesure de la réponse.
11	Principal avantage du monolithe modulaire sur les microservices (Quiz 3) ?	**B**	Découplage logique strict, sans la complexité réseau et financière des microservices.
12	Les constantes vitales d'un patient en temps réel : synchrone ou asynchrone ?	**Asynchrone**	Flux continu, plusieurs consommateurs (alertes, dossier, IA), tolérance aux pannes (voir sujet 3).

Sujets transversaux type DS (corrigés)

SUJETS D'ENTRAÎNEMENT CONÇUS POUR LA RÉVISION · PROPOSITION DE CORRIGÉ — NON FOURNIE DANS LE COURS

Sujet 1 — Revue de code (ch. 1 + ch. 2). On donne la classe suivante.

```
public class NotificationManager {
    private EmailSender email = new EmailSender();
    private SmsGateway  sms   = new SmsGateway();      // API différente : send(num, txt)
    public void notifier(String canal, String dest, String msg) {
        if (canal.equals("EMAIL"))     email.envoyer(dest, msg);
        else if (canal.equals("SMS"))  sms.send(dest, msg);
        // + "PUSH" la semaine prochaine...
    }
    public void journaliser(String msg) { /* écrit dans un fichier */ }
}
```

a) Quels principes SOLID sont violés ? b) Quels patrons du ch. 1 proposer ? c) Donner le code corrigé et le schéma de classes.

CORRIGÉ

a) Trois principes sont violés :

- **OCP** : chaque nouveau canal (PUSH) oblige à modifier `notifier()` et sa chaîne de `if`.
- **DIP** : la classe de haut niveau crée elle-même des classes concrètes (`new EmailSender()`).
- **SRP** : elle notifie *et* journalise, ce qui fait deux raisons de changer.

Risques visibles : **rigidité** et **immobilité**. **b)** Quatre patrons répondent au problème :

- Une interface `Canal` (polymorphisme / **Stratégie**), qui règle l'OCP.

- Un **Adaptateur** `SmsAdapter`, car `SmsGateway` a une API incompatible (`send` au lieu de `envoyer`).
- Une **Fabrique** qui crée le canal à partir de la chaîne, et elle-même en **Singleton**.
- Le journal est extrait dans une classe `Journal` (SRP).

```
public interface Canal { void envoyer(String dest, String msg); }

public class EmailCanal implements Canal { public void envoyer(String d, String m) { /* SMTP */ } }
public class SmsAdapter implements Canal {              // Adaptateur
    private final SmsGateway gateway = new SmsGateway(); // Adaptee
    public void envoyer(String dest, String msg) { gateway.send(dest, msg); }
}
public final class CanalFactory {                       // Fabrique + Singleton
    private static CanalFactory instance;
    private CanalFactory() { }
    public static synchronized CanalFactory getInstance() {
        if (instance == null) instance = new CanalFactory();
        return instance;
    }
    public Canal creer(String type) {
        switch (type) {
            case "EMAIL": return new EmailCanal();
            case "SMS":   return new SmsAdapter();
            default: throw new IllegalArgumentException(type);
        }
    }
}
public class NotificationService {                      // haut niveau
    private final Canal canal;                          // dépend de l'abstraction
    public NotificationService(Canal canal) { this.canal = canal; }
    public void notifier(String dest, String msg) { canal.envoyer(dest, msg); }
}
```

c) Schéma : `NotificationService` → «use» `«interface» Canal` ; `EmailCanal`, `SmsAdapter` ⇢▷ `Canal` ; `SmsAdapter` → `SmsGateway` ; `CanalFactory` ⇢ «create» les canaux concrets. Ajouter PUSH = une classe `PushCanal` + un `case` dans la fabrique ; le service ne change pas (le `switch` reste **confiné dans la fabrique**).

Sujet 2 — Design ou architecture ? (ch. 3). Classer chaque décision et justifier par les 4 critères : (a) utiliser une `HashMap` plutôt qu'une `TreeMap` dans une méthode ; (b) remplacer les appels REST entre Commandes et Stock par des événements Kafka ; (c) héberger les dossiers patients chez un hébergeur certifié HDS ; (d) appliquer le patron Stratégie au calcul des frais de port ; (e) partager une seule base de données entre trois services.

CORRIGÉ

- (a) **Design** : une seule méthode, retour arrière en une ligne, aucun attribut de qualité global.
- (b) **Architecture** : plusieurs services, retour coûteux (contrats, infrastructure), performance et disponibilité changent. Trois « oui ».
- (c) **Architecture** : les parties prenantes sont multiples (juridique RGPD/HDS, exploitation, coût) et la sécurité est en jeu.
- (d) **Design** (micro-architecture) : quelques classes, retour arrière facile.
- (e) **Architecture** : c'est la V1 du cours. Couplage fort entre services, goulot d'étranglement, et des données à migrer pour en sortir.

Sujet 3 — Synchrone, asynchrone et Observateur (ch. 1 + ch. 3). Pour ShopExpress, choisir un mode de communication pour (a) l'autorisation du paiement et (b) l'envoi de l'e-mail de confirmation, la mise à jour des recommandations et la mise à jour du stock après une commande. Quel patron du chapitre 1 est la version « dans le code » de la solution (b) ?

CORRIGÉ

(a) Synchrone (REST/gRPC avec délai maximal) : le client doit connaître immédiatement le résultat, et la cohérence est forte. On accepte le couplage temporel sur ce seul appel critique.

(b) Asynchrone : Commandes publie un événement `CommandeValidée` sur un bus (Kafka/RabbitMQ) ; e-mail, recommandations et stock s'y abonnent. Gains : découplage temporel, tolérance aux pannes (l'e-mail en panne ne bloque pas la commande), ajout d'un abonné sans toucher à l'émetteur (OCP). Prix : cohérence éventuelle, orchestration plus complexe.

Dans le code, c'est l'**Observateur** : le sujet notifie tous ses observateurs (`notifyAllObservers()` → `update()`) ; le bus d'événements le généralise entre services.